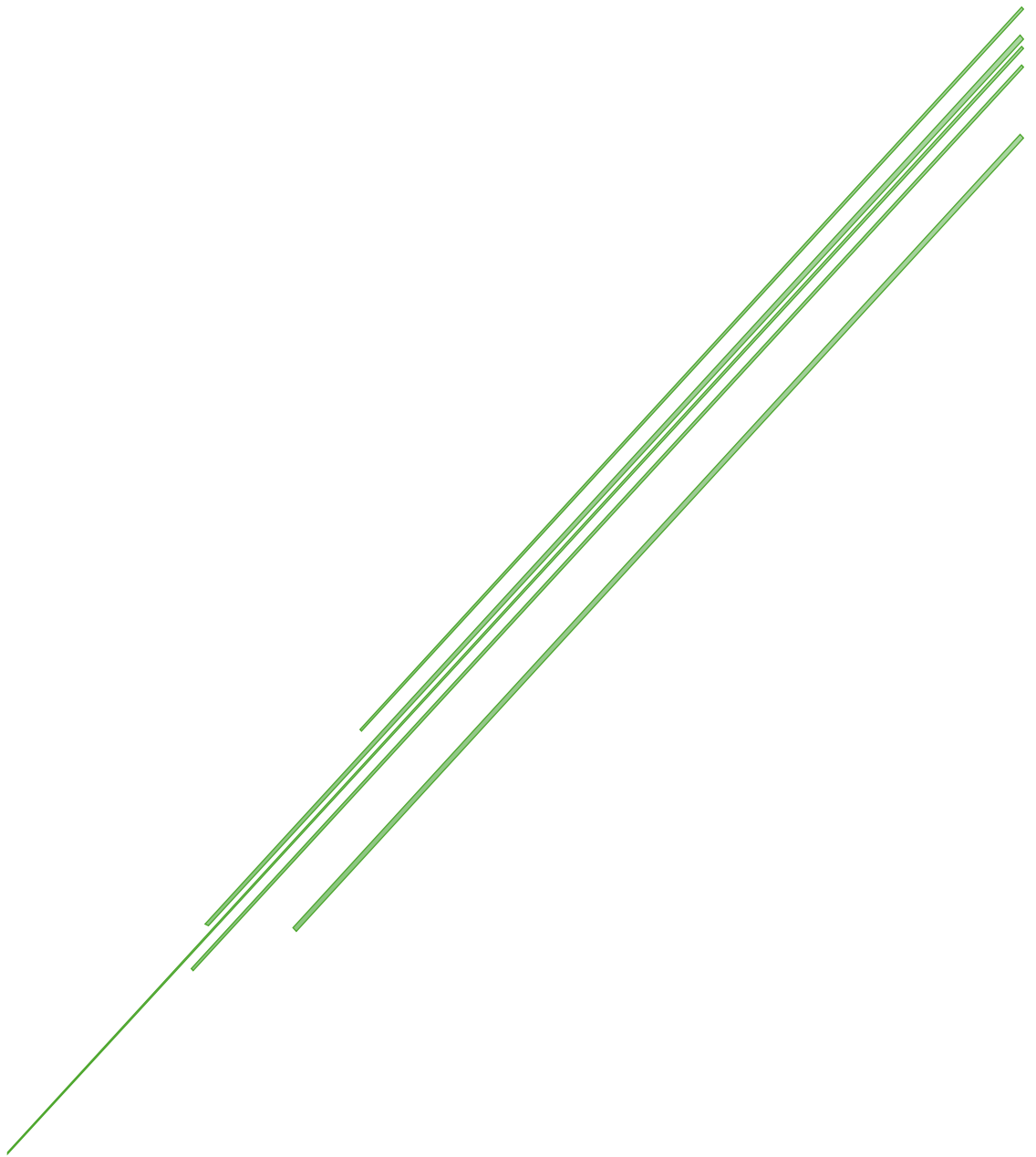


# SUPER CRUD EM MEMÓRIA

Java



## Sumário

|                                                           |    |
|-----------------------------------------------------------|----|
| CRUD em memória .....                                     | 2  |
| Salvar .....                                              | 11 |
| Atualizar .....                                           | 11 |
| Listar .....                                              | 11 |
| Remover .....                                             | 12 |
| FindById.....                                             | 12 |
| ProgramaPrincipal.....                                    | 14 |
| Cadastrar.....                                            | 16 |
| Alterar.....                                              | 17 |
| Listar .....                                              | 18 |
| Remover .....                                             | 19 |
| Código de “ProgramaPrincipal” .....                       | 20 |
| Ajustes finais .....                                      | 22 |
| Números mágicos .....                                     | 29 |
| Código final completo da classe “ProgramaPrincipal” ..... | 35 |
| GitHub.....                                               | 37 |
| Testes unitários.....                                     | 38 |
| Introdução .....                                          | 38 |
| Implementação.....                                        | 39 |
| Cenários .....                                            | 43 |
| @BeforeEach .....                                         | 47 |
| Métodos assert .....                                      | 48 |
| Teste do cadastro .....                                   | 50 |
| Teste de alteração de usuário .....                       | 52 |
| Teste listar usuários.....                                | 53 |
| Teste remover usuário .....                               | 54 |
| Classe Console.....                                       | 56 |

# CRUD em memória

CRUD é um acrônimo que representa as quatro operações básicas realizadas em bancos de dados e sistemas de informação:

- **C – Create** (Criar): Inserir novos dados no sistema.
- **R – Read** (Ler): Consultar ou visualizar dados existentes.
- **U – Update** (Atualizar): Modificar dados já existentes.
- **D – Delete** (Deletar): Remover dados do sistema.

## Exemplo simples:

Imagina um sistema de cadastro de usuários. O CRUD seria:

- **Create:** Cadastrar um novo usuário.
- **Read:** Listar os usuários cadastrados.
- **Update:** Alterar os dados de um usuário (como o nome ou e-mail).
- **Delete:** Excluir um usuário do sistema.

---

## Importância do CRUD no desenvolvimento de sistemas:

1. **Base para qualquer aplicação:** A maioria dos sistemas precisa gerenciar dados (clientes, produtos, pedidos, etc.), e o CRUD é a forma básica de fazer isso.
2. **Organização do código:** Seguir essa estrutura ajuda a manter o código organizado, com funções bem definidas para cada operação.
3. **Facilidade de manutenção:** Quando o sistema é desenvolvido com base em operações CRUD, fica mais fácil fazer ajustes e melhorias no futuro.
4. **Interface com banco de dados:** O CRUD representa as interações fundamentais com qualquer banco de dados relacional ou não relacional.
5. **Reutilização e padronização:** Como é uma prática comum, os desenvolvedores conseguem entender e trabalhar em sistemas de outras pessoas com mais facilidade.

O CRUD está diretamente **relacionado ao banco de dados**, porque ele define as **quatro operações básicas** que você faz **com os dados armazenados** lá. Vamos entender melhor essa relação:



### Banco de dados:

É onde os dados do sistema ficam guardados — como uma “caixa” que armazena informações sobre usuários, produtos, pedidos, etc.

---

## **CRUD no contexto do banco de dados:**

### **1. CREATE (Criar):**

- Operação que insere novos registros no banco.
- Exemplo SQL:

```
INSERT INTO tb_usuarios (nome, email) VALUES ('Maria', 'maria@email.com');
```

### **2. READ (Ler):**

- Consulta os dados no banco, seja para exibir ou processar.
- Exemplo SQL:

```
SELECT * FROM tb_usuarios;
```

### **3. UPDATE (Atualizar):**

- Modifica dados existentes.
- Exemplo SQL:

```
UPDATE tb_usuarios SET nome = 'Maria Silva' WHERE id = 1;
```

### **4. DELETE (Deletar):**

- Remove registros do banco de dados.
- Exemplo SQL:
- 

```
DELETE FROM tb_usuarios WHERE id = 1;
```

---

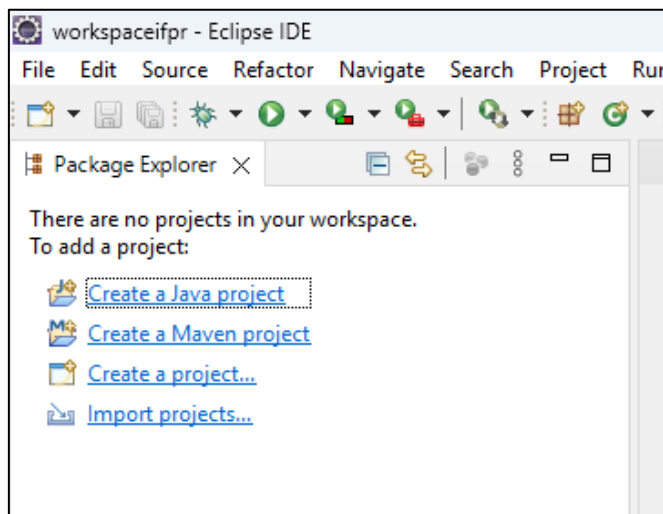
### **Resumindo:**

O **CRUD** é como o "vocabulário básico" de comunicação com um banco de dados. Toda vez que um sistema precisa armazenar, mostrar, alterar ou apagar dados, ele faz isso através de operações CRUD.

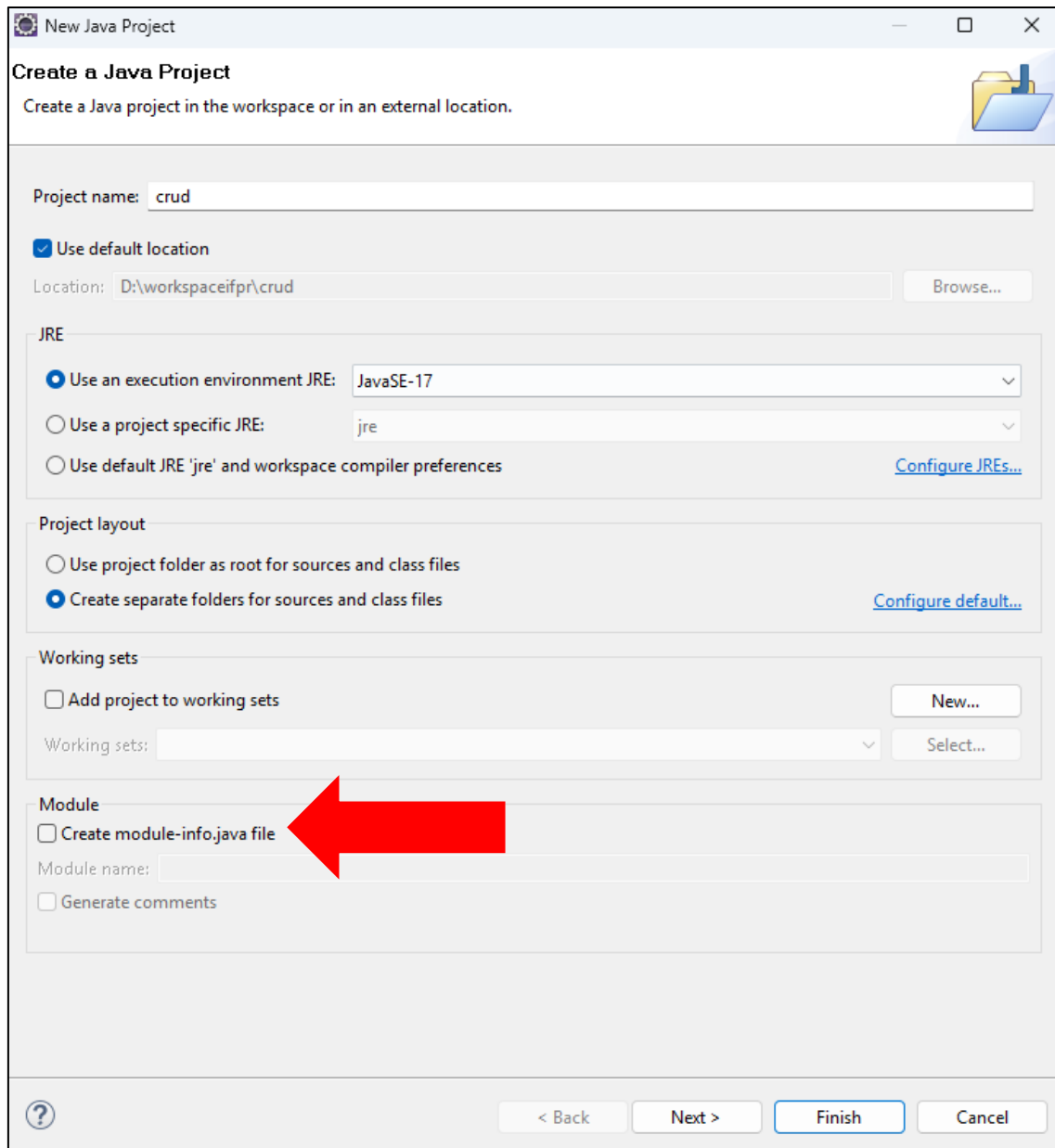
Essas operações podem ser feitas manualmente (com SQL, por exemplo) ou através de linguagens de programação (como Python, JavaScript, Java) que se conectam ao banco.

Dada essa introdução, vamos iniciar nosso o projeto.

O primeiro passo é criar um projeto java:



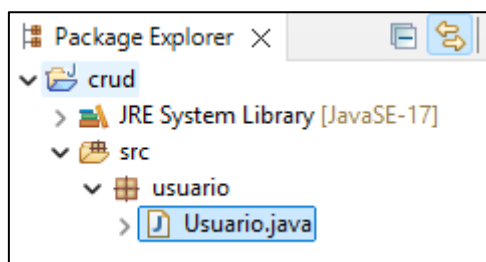
Coloque o nome do projeto como “crud” e desmarque a caixa “Create module-info.java file” e clique em “Finish”:



Vamos fazer o crud de uma classe com 4 atributos:

- Usuario (id, nome, cpf, dataNascimento)

Crie um pacote chamado “usuario” e dentro crie a classe “Usuario”:



```
package usuario;

import java.time.LocalDate;

public class Usuario {

    private int id;
    private String nome;
    private long cpf;
    private LocalDate dataNascimento;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public long getCpf() {
        return cpf;
    }

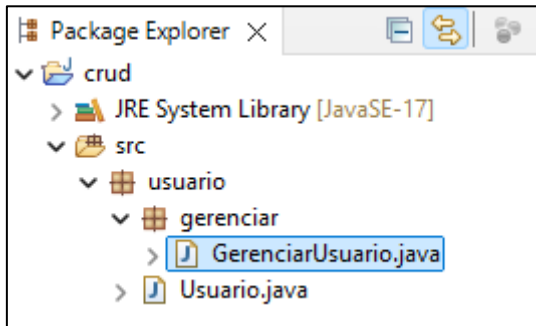
    public void setCpf(long cpf) {
        this.cpf = cpf;
    }

    public LocalDate getDataNascimento() {
        return dataNascimento;
    }

    public void setDataNascimento(LocalDate dataNascimento) {
        this.dataNascimento = dataNascimento;
    }

}
```

Dentro do pacote “usuario” crie outro pacote com o nome “gerenciar”. Dentro do pacote “gerenciar” crie uma classe com o nome “GerenciarUsuario”. Essa classe será responsável por manipular a entidade no banco de dados, no nosso caso, o banco de dados será uma estrutura de dados em memória (HashMap).



Vamos falar de **HashMap** em Java — um dos recursos mais usados quando você precisa **associar uma chave a um valor** (como um dicionário ou um mapa).

---

### O que é um HashMap?

O HashMap é uma **estrutura de dados da linguagem Java** que armazena **pares chave-valor**.

- **Chave (key):** valor único usado para identificar um item.
- **Valor (value):** dado associado à chave.

Por exemplo:

```
HashMap<String, String> capitais = new HashMap<>();

capitais.put("Brasil", "Brasília");
capitais.put("Argentina", "Buenos Aires");
capitais.put("Chile", "Santiago");
```

Nesse caso, o país é a **chave** e a capital é o **valor**.

---

### Como funciona por trás dos panos?

#### 1. Hashing:

- Quando você adiciona uma chave, o Java usa a função hashCode() da chave para gerar um número.
- Esse número é mapeado para um **índice** interno (posição na memória).
- Esse índice define onde o valor será armazenado.

#### 2. Colisões:

- Às vezes, duas chaves diferentes geram o mesmo índice. Isso é chamado de **colisão**.
- O HashMap lida com isso usando **listas encadeadas** ou **árvores balanceadas** (em versões mais novas do Java).

#### 3. Busca eficiente:

- Como o acesso é feito pelo índice gerado pelo hashCode, o tempo de busca é **muito rápido** (complexidade média: O(1)).



## Operações básicas:

```
import java.util.HashMap;

public class ExemploHashMap {
    public static void main(String[] args) {
        HashMap<String, Integer> estoque = new HashMap<>();

        // Adicionando itens
        estoque.put("Camiseta", 10);
        estoque.put("Tênis", 5);
        estoque.put("Calça", 8);

        // Acessando valores
        System.out.println("Quantidade de camisetas: " +
estoque.get("Camiseta"));

        // Verificando se contém uma chave
        if (estoque.containsKey("Tênis")) {
            System.out.println("Tem tênis no estoque.");
        }

        // Removendo item
        estoque.remove("Calça");

        // Iterando sobre o mapa
        for (String item : estoque.keySet()) {
            System.out.println(item + ": " + estoque.get(item));
        }
    }
}
```

## Cuidado com:

- null como chave é permitido (apenas uma vez).
- null como valor é permitido várias vezes.
- Não é sincronizado por padrão (não é thread-safe).

## HashMap vs outras estruturas em Java

| Estrutura     | Ordenação das chaves                       | Permite <code>null</code> ?     | É sincronizada (thread-safe)? |
|---------------|--------------------------------------------|---------------------------------|-------------------------------|
| HashMap       | ✗ Não mantém ordem                         | ✓ 1 chave, vários valores       | ✗ Não                         |
| Hashtable     | ✗ Não mantém ordem                         | ✗ Não permite <code>null</code> | ✓ Sim                         |
| TreeMap       | ✓ Ordena por chave (natural ou Comparator) | ✓ Chaves e valores              | ✗ Não                         |
| LinkedHashMap | ✓ Mantém a ordem de inserção               | ✓                               | ✗ Não                         |

### Analogia: Mochila com etiquetas

Imagina que você tem uma **mochila cheia de compartimentos** (como os espaços internos de um HashMap).

Cada compartimento tem uma **etiqueta (chave)** e guarda **algum item (valor)**.

- Quando você quer guardar algo, você cola uma etiqueta (ex: "Fone de Ouvido") em um compartimento e coloca o item lá.
- Quando você quer achar esse item depois, você **procura direto pela etiqueta**, sem ter que abrir tudo.
- Se duas etiquetas gerarem a mesma posição interna (colisão), você põe ambas as etiquetas naquele mesmo lugar, mas de um jeito organizado (lista ou árvore).
- Se você remover uma etiqueta, o item associado a ela também sai.

👉 Esse é o conceito do HashMap: **guardar e acessar valores rapidamente com base em uma chave única**.

---

### Quando usar cada um?

- **HashMap:** Quando você precisa de performance e **não se importa com a ordem**.
- **TreeMap:** Quando a **ordem importa** (ex: lista de nomes em ordem alfabética).
- **LinkedHashMap:** Quando você quer **manter a ordem em que inseriu** os dados.
- **Hashtable:** Quase nunca hoje em dia — foi substituído por estruturas mais modernas como ConcurrentHashMap.

Agora vamos criar os seguintes métodos dentro da classe “GerenciarUsuario”:

- **public void salvar(Usuario usuario)**
- **public void atualizar(Usuario usuário)**
- **public List<Usuario> listar()**
- **public void remover(int id)**

Repare que esses métodos representam o CRUD:

- salvar (C – create)
- listar (R – read)
- atualizar (U – update)
- remover (D – delete)

Além dos métodos do CRUD, vamos colocar mais um método auxiliar:

- **public Usuario findById(int id)**

Esse método irá nos ajudar na implementação da edição de um usuário (atualizar).

Por enquanto o código da “GerenciarUsuario” deverá estar assim:

```
package usuario.gerenciar;

import java.util.List;

import usuario.Usuario;

public class GerenciarUsuario {

    public void salvar(Usuario usuario) {

    }

    public void atualizar(Usuario usuario) {

    }

    public List<Usuario> listar() {
        return null;
    }

    public void remover(int id) {

    }

    public Usuario findById(int id) {
        return null;
    }

}
```

Vamos criar dois atributos, um será o nosso banco de dados e o outro irá controlar os IDs dos usuários:

```
private HashMap<Integer, Usuario> usuariosBD;
private int id;
```

Crie um método construtor e inicialize os dois atributos acima:

```
public GerenciarUsuario() {
    usuariosBD = new HashMap<>();
    id = 0;
}
```

## Salvar

O método salvar deve gerar um novo id, definir esse id no objeto usuário e adicionar a chave e o valor no HashMap:

```
public void salvar(Usuario usuario) {  
    id++;  
    usuario.setId(id);  
    usuariosBD.put(id, usuario);  
}
```

## Atualizar

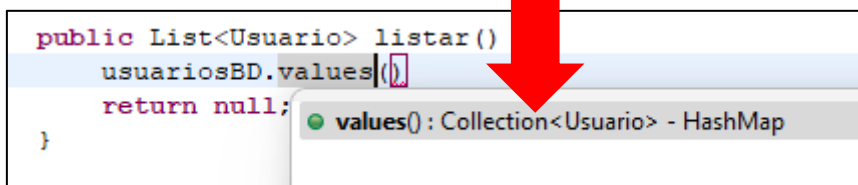
O método atualizar deve apenas substituir o objeto antigo pelo novo no HashMap, para isso basta fazer um “put” novamente com a chave do usuário existente (id):

```
public void atualizar(Usuario usuario) {  
    usuariosBD.put(usuario.getId(), usuario);  
}
```

## Listar

O método listar deve retornar uma lista com todos os usuários do banco de dados. No nosso caso, estamos usando um HashMap, porém, queremos somente a informação armazenada em “value” do HashMap, não queremos as chaves.

O HashMap possui um método que retorna os valores, porém, a o retorno é uma Collection<Usuario>:



Devemos converter essa coleção para uma lista de usuários, existem algumas maneiras de fazer isso:

- podemos criar uma lista vazia, fazer um loop nos valores e adicionar os elementos manualmente dentro da lista e por fim, retornar a lista com os usuários:

```
public List<Usuario> listar() {  
    List<Usuario> usuarios = new ArrayList<>();  
    for (Usuario usuario : usuariosBD.values()) {  
        usuarios.add(usuario);  
    }  
    return usuarios;  
}
```

Ou podemos passar como parâmetro do construtor de ArrayList, a coleção de valores do HashMap:

```
public List<Usuario> listar() {  
    List<Usuario> usuarios = new ArrayList<>(usuariosBD.values());  
    return usuarios;  
}
```

Vamos essa segunda maneira de criar a lista de usuários.

## Remover

O método remover deve remover a chave e o valor do HashMap, para isto basta informar a chave que deve ser removida:

```
public void remover(int id) {  
    usuariosBD.remove(id);  
}
```

## FindById

O método findById deve buscar um usuário pelo id dentro do HashMap, para isso basta usar o método “get” do HashMap:

```
public Usuario findById(int id) {  
    return usuariosBD.get(id);  
}
```

Segue o código completo de “GerenciarUsuario”:

```
package usuario.gerenciar;  
  
import java.util.ArrayList;  
import java.util.HashMap;  
import java.util.List;  
  
import usuario.Usuario;  
  
public class GerenciarUsuario {  
  
    private HashMap<Integer, Usuario> usuariosBD;  
    private int id;  
  
    public GerenciarUsuario() {  
        usuariosBD = new HashMap<>();  
        id = 0;  
    }  
}
```

```
}

public void salvar(Usuario usuario) {
    id++;
    usuario.setId(id);
    usuariosBD.put(id, usuario);
}

public void atualizar(Usuario usuario) {
    usuariosBD.put(usuario.getId(), usuario);
}

public List<Usuario> listar() {
    List<Usuario> usuarios = new ArrayList<>(usuariosBD.values());
    return usuarios;
}

public void remover(int id) {
    usuariosBD.remove(id);
}

public Usuario findById(int id) {
    return usuariosBD.get(id);
}

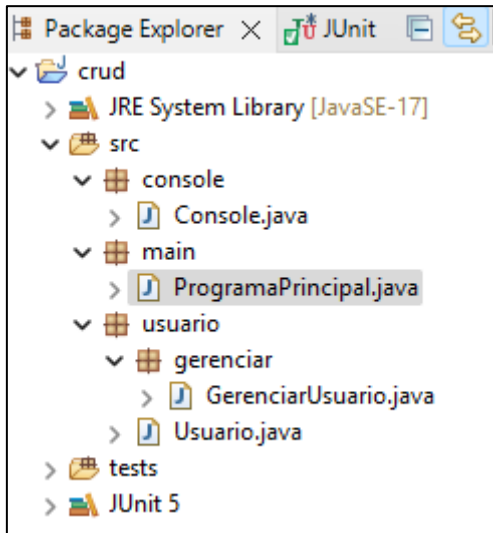
}
```

## ProgramaPrincipal

Vamos iniciar a implementação do nosso programa, vamos juntar todas as peças e fazê-las funcionar.

Como nosso programa vai funcionar em modo texto (cmd), vamos usar a classe auxiliar “Console” para fazer a leitura do teclado. O código está no fim desse documento.

Crie um pacote chamado “main” e dentro crie a classe “ProgramaPrincipal” com o método main:



O programa deverá mostrar o seguinte menu:

```
--- SUPER CRUD ---  
1 – Cadastrar usuário  
2 – Alterar usuário  
3 – Listar usuários  
4 – Remover Usuário  
9 – Sair
```

O programa deve permanecer em loop até que o usuário escolha a opção 9. Para isso usaremos um loop **do...while**, pois o menu deve ser mostrado pelo menos uma vez durante a execução do programa.

Vamos iniciar declarando uma variável para a opção escolhida pelo usuário, uma variável para a classe que manipula a entidade no banco de dados (GerenciarUsuario) e uma para ler do teclado usando a classe Console:

```
package main;  
  
import console.Console;  
  
public class ProgramaPrincipal {
```

```
public static void main(String[] args) {
    GerenciarUsuario gerenciadorUsuario = new GerenciarUsuario();
    Console console = new Console();
    int opcao = 0;

    do {

        System.out.println("");
        System.out.println("--- SUPER CRUD ---");
        System.out.println("");
        System.out.println("1 - Cadastrar usuário");
        System.out.println("2 - Alterar usuário");
        System.out.println("3 - Listar usuários");
        System.out.println("4 - Remover usuário");
        System.out.println("9 - Sair");

        opcao = console.readInt("Digite uma opção: ");

        if (opcao == 1) {

        } else if (opcao == 2) {

        } else if (opcao == 3) {

        } else if (opcao == 4) {

        }

    } while (opcao != 9);

    System.out.println("Terminando o programa, bye");
}
}
```



## Cadastrar

A lógica do cadastro deve ser a seguinte:

- 1 – Criar um novo objeto da classe Usuario.
- 2 – Ler os dados do usuário (nome, cpf e data de nascimento), lembre-se que o ID deve ser gerado pelo banco de dados.
- 3 – Definir os atributos do Usuario com os dados lidos do teclado.
- 4 – Chamar o `gerenciarUsuario` para salvar o novo objeto no banco de dados.
- 5 – Mostrar mensagem de sucesso para o usuário.

O código ficará dessa maneira:

```
if (opcao == 1) {  
    System.out.println("\nCadastrar novo usuário\n");  
  
    Usuario usuario = new Usuario();  
  
    String nome = console.readLine("Digite o nome: ");  
    long cpf = console.readLong("Digite o CPF: ");  
    LocalDate dataNascimento = console.readLocalDate("Digite a data de nascimento: ");  
  
    usuario.setNome(nome);  
    usuario.setCpf(cpf);  
    usuario.setDataNascimento(dataNascimento);  
  
    gerenciarUsuario.salvar(usuario);  
  
    System.out.println("\nUsuário criado com sucesso\n");  
}
```

## Alterar

A lógica da alteração deve ser a seguinte:

- 1 – Perguntar para o usuário qual ID ele quer editar (caso o usuário não saiba, ele deve listar os usuários e descobrir o ID, ou mais para o fim desse documento iremos implementar uma pesquisa pelo nome do usuário).
- 2 – Buscar o objeto referente ao ID escolhido no banco de dados para fazermos as alterações.
- 3 – Ler novamente os dados do usuário (nome, cpf e data de nascimento).
- 4 – Definir os atributos do Usuario com os novos dados lidos do teclado.
- 5 – Chamar o `gerenciarUsuario` para atualizar objeto no banco de dados.
- 6 – Mostrar mensagem de sucesso para o usuário.

O código ficará dessa maneira:

```
} else if (opcao == 2) {  
    System.out.println("\nAlterar usuário\n");  
  
    int idParaAlterar = console.readInt("Digite o ID para alterar: ");  
  
    Usuario usuario = gerenciarUsuario.findById(idParaAlterar);  
  
    String nome = console.readLine("Digite o nome: ");  
    long cpf = console.readLong("Digite o CPF: ");  
    LocalDate dataNascimento = console.readLocalDate("Digite a data de nascimento: ");  
  
    usuario.setNome(nome);  
    usuario.setCpf(cpf);  
    usuario.setDataNascimento(dataNascimento);  
  
    gerenciarUsuario.atualizar(usuario);  
  
    System.out.println("\nUsuário alterado com sucesso\n");  
}
```

## Listar

A lógica para listar os usuários é a seguinte:

- 1 – Criar uma lista de usuário e já atribuir o retorno da chamada do método listar da classe gerenciarUsuario.
- 2 – Fazer um loop na lista de usuários (pode ser um **for** ou um **foreach**).
- 3 – Mostrar na tela os dados de cada elemento da lista.

O código ficará dessa maneira:

```
} else if (opcao == 3) {  
    System.out.println("\nListar usuários\n");  
    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");  
    List<Usuario> usuarios = gerenciarUsuario.listar();  
    for (Usuario usuario : usuarios) {  
        System.out.println("\nID: " + usuario.getId());  
        System.out.println("Nome: " + usuario.getNome());  
        System.out.println("CPF: " + usuario.getCpf());  
        System.out.println("Data      de      nascimento:      "      +  
dtf.format(usuario.getDataNascimento()));  
    }  
}
```

## Remover

A lógica do remover deve ser a seguinte:

- 1 – Perguntar para o usuário qual ID ele quer remover (caso o usuário não saiba, ele deve listar os usuários e descobrir o ID, ou mais para o fim desse documento iremos implementar uma pesquisa pelo nome do usuário).
- 2 – Chamar o gerenciar e passar como parâmetro do método remover o id escolhido.
- 3 – Mostrar uma mensagem de sucesso na tela.

O código ficará dessa maneira:

```
} else if (opcao == 4) {  
    System.out.println("\nRemover usuário\n");  
  
    int idParaExcluir = console.readInt("Digite o ID para excluir: ");  
  
    gerenciarUsuario.remover(idParaExcluir);  
    System.out.println("\nUsuário excluído com sucesso");  
}
```

## Código de “ProgramaPrincipal”

```
package main;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;

import console.Console;
import usuario.Usuario;
import usuario.gerenciar.GerenciarUsuario;

public class ProgramaPrincipal {

    public static void main(String[] args) {
        GerenciarUsuario gerenciarUsuario = new GerenciarUsuario();
        Console console = new Console();
        int opcao = 0;

        do {

            System.out.println("");
            System.out.println("--- SUPER CRUD ---");
            System.out.println("");
            System.out.println("1 - Cadastrar usuário");
            System.out.println("2 - Alterar usuário");
            System.out.println("3 - Listar usuários");
            System.out.println("4 - Remover usuário");
            System.out.println("9 - Sair");

            opcao = console.readInt("Digite uma opção: ");

            if (opcao == 1) {
                System.out.println("\nCadastrar novo usuário\n");

                Usuario usuario = new Usuario();

                String nome = console.readLine("Digite o nome: ");
                long cpf = console.readLong("Digite o CPF: ");
                LocalDate dataNascimento = console.readLocalDate("Digite a
data de nascimento: ");

                usuario.setNome(nome);
                usuario.setCpf(cpf);
                usuario.setDataNascimento(dataNascimento);

                gerenciarUsuario.salvar(usuario);

                System.out.println("\nUsuário criado com sucesso\n");
            } else if (opcao == 2) {
                System.out.println("\nAlterar usuário\n");
```

```

        int idParaAlterar = console.readInt("Digite o ID para alterar: ");

        Usuario usuario = gerenciarUsuario.findByld(idParaAlterar);

        String nome = console.readLine("Digite o nome: ");
        long cpf = console.readLong("Digite o CPF: ");
        LocalDate dataNascimento = console.readLocalDate("Digite a
data de nascimento: ");

        usuario.setNome(nome);
        usuario.setCpf(cpf);
        usuario.setDataNascimento(dataNascimento);

        gerenciarUsuario.atualizar(usuario);

        System.out.println("\nUsuário alterado com sucesso\n");
    } else if (opcao == 3) {
        System.out.println("\nListar usuários\n");
        DateTimeFormatter dtf =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
        List<Usuario> usuarios = gerenciarUsuario.listar();
        for (Usuario usuario : usuarios) {
            System.out.println("ID: " + usuario.getId());
            System.out.println("Nome: " + usuario.getNome());
            System.out.println("CPF: " + usuario.getCpf());
            System.out.println("Data de nascimento: " +
dtf.format(usuario.getDataNascimento()));
        }
    } else if (opcao == 4) {
        System.out.println("\nRemover usuário\n");

        int idParaExcluir = console.readInt("Digite o ID para excluir: ");

        gerenciarUsuario.remover(idParaExcluir);
        System.out.println("\nUsuário excluído com sucesso");
    }
} while (opcao != 9);

System.out.println("Terminando o programa, bye");
}
}

```

## Ajustes finais

Temos um CRUD completo implementado em “ProgramaPrincipal”, porém, não está orientado a objetos. Fizemos tudo dentro da main e temos alguns problemas, como números mágicos e código duplicado.

Primeiro vamos colocar a console e a variável `gerenciarUsuario` como atributos da classe:

```
private GerenciarUsuario gerenciarUsuario;  
private Console console;
```

Apenas declare os atributos, a inicialização dos atributos é sempre feita no método construtor, então, crie o método construtor e inicialize os dois atributos:

```
public ProgramaPrincipal() {  
    gerenciarUsuario = new GerenciarUsuario();  
    console = new Console();  
}
```

Mude o nome do método “**main**” para que fique dessa maneira:

```
private void executar() {  
    int opcao = 0;  
    do {  
  
        System.out.println("");  
        System.out.println("--- SUPER CRUD ---");  
        System.out.println("");  
        // restante do código
```

Crie novamente um método “**main**” da seguinte maneira:

```
public static void main(String[] args) {  
    new ProgramaPrincipal().executar();  
}
```

Por enquanto temos o código dessa maneira:

```
package main;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;

import console.Console;
import usuario.Usuario;
import usuario.gerenciar.GerenciarUsuario;

public class ProgramaPrincipal {

    private GerenciarUsuario gerenciarUsuario;
    private Console console;

    public ProgramaPrincipal() {
        gerenciarUsuario = new GerenciarUsuario();
        console = new Console();
    }

    public static void main(String[] args) {
        new ProgramaPrincipal().executar();
    }

    private void executar() {

        int opcao = 0;

        do {

            System.out.println("");
            System.out.println("--- SUPER CRUD ---");
            System.out.println("");
            System.out.println("1 - Cadastrar usuário");
            System.out.println("2 - Alterar usuário");
            System.out.println("3 - Listar usuários");
            System.out.println("4 - Remover usuário");
            System.out.println("9 - Sair");

            opcao = console.readInt("Digite uma opção: ");

            if (opcao == 1) {

                System.out.println("\nCadastrar novo usuário\n");

                Usuario usuario = new Usuario();

                String nome = console.readLine("Digite o nome: ");
                long cpf = console.readLong("Digite o CPF: ");
                LocalDate dataNascimento = console.readLocalDate("Digite a
data de nascimento: ");

                usuario.setNome(nome);
                usuario.setCpf(cpf);
```



```

        usuario.setDataNascimento(dataNascimento);

        gerenciarUsuario.salvar(usuario);

        System.out.println("\nUsuário criado com sucesso\n");

    } else if (opcao == 2) {
        System.out.println("\nAlterar usuário\n");

        int idParaAlterar = console.readInt("Digite o ID para alterar: ");

        Usuario usuario = gerenciarUsuario.findById(idParaAlterar);

        String nome = console.readLine("Digite o nome: ");
        long cpf = console.readLong("Digite o CPF: ");
        LocalDate dataNascimento = console.readLocalDate("Digite a
data de nascimento: ");

        usuario.setNome(nome);
        usuario.setCpf(cpf);
        usuario.setDataNascimento(dataNascimento);

        gerenciarUsuario.atualizar(usuario);

        System.out.println("\nUsuário alterado com sucesso\n");
    } else if (opcao == 3) {
        System.out.println("\nListar usuários\n");
        DateTimeFormatter dtf =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
        List<Usuario> usuarios = gerenciarUsuario.listar();
        for (Usuario usuario : usuarios) {
            System.out.println("\nID: " + usuario.getId());
            System.out.println("Nome: " + usuario.getNome());
            System.out.println("CPF: " + usuario.getCpf());
            System.out.println("Data de nascimento: " +
dtf.format(usuario.getDataNascimento()));
        }
    } else if (opcao == 4) {
        System.out.println("\nRemover usuário\n");

        int idParaExcluir = console.readInt("Digite o ID para excluir: ");

        gerenciarUsuario.remover(idParaExcluir);
        System.out.println("\nUsuário excluído com sucesso");
    }
}

} while (opcao != 9);

System.out.println("Terminando o programa, bye");
}
}

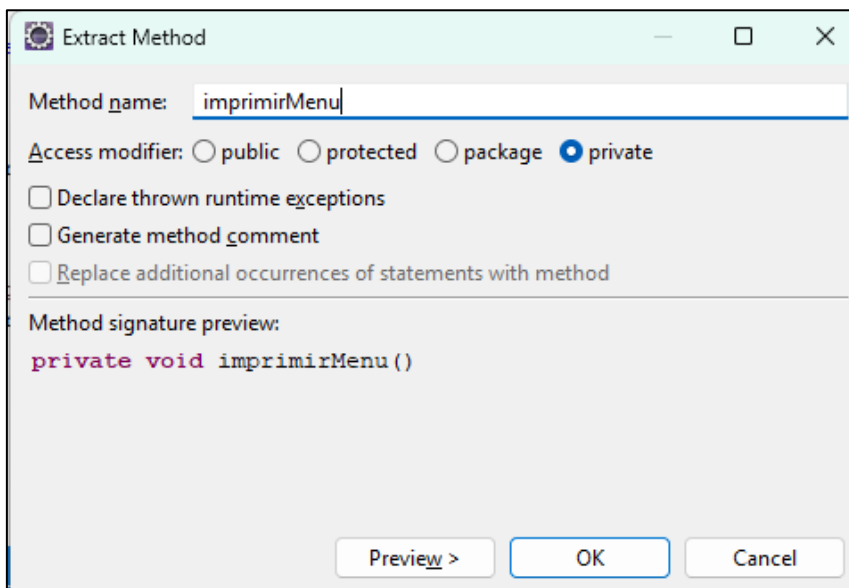
```

Agora vamos refatorar o código em métodos menores com menos responsabilidade (cada método com uma responsabilidade).

Selecione a parte do código que está sendo usada para mostrar o menu na tela:

```
25 private void executar() {  
26  
27     int opcao = 0;  
28  
29     do {  
30         System.out.println("");  
31         System.out.println("--- SUPER CRUD ---");  
32         System.out.println("");  
33         System.out.println("1 - Cadastrar usuário");  
34         System.out.println("2 - Alterar usuário");  
35         System.out.println("3 - Listar usuários");  
36         System.out.println("4 - Remover usuário");  
37         System.out.println("9 - Sair");  
38     }
```

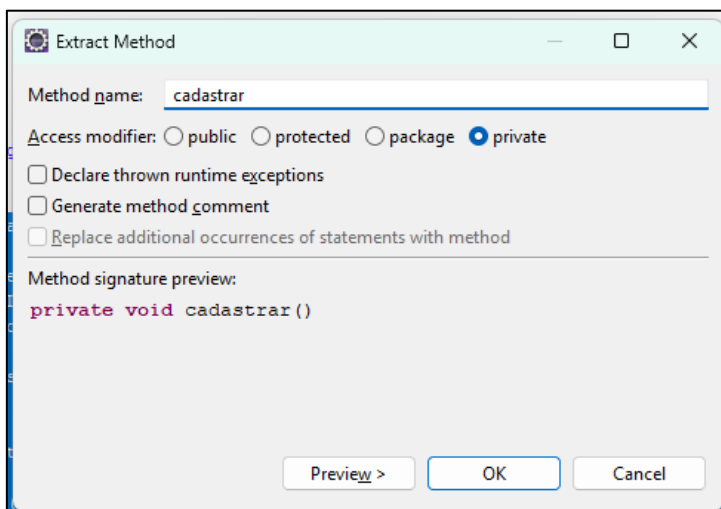
Aperte ALT + SHIFT + M para extrair esse bloco de código para um método com o nome “imprimirMenu”:



O corpo de cada IF será refatorado para métodos referentes às operações que eles realizam, assim, o primeiro IF serve para cadastrar um novo usuário, o segundo para alterar e assim por diante.

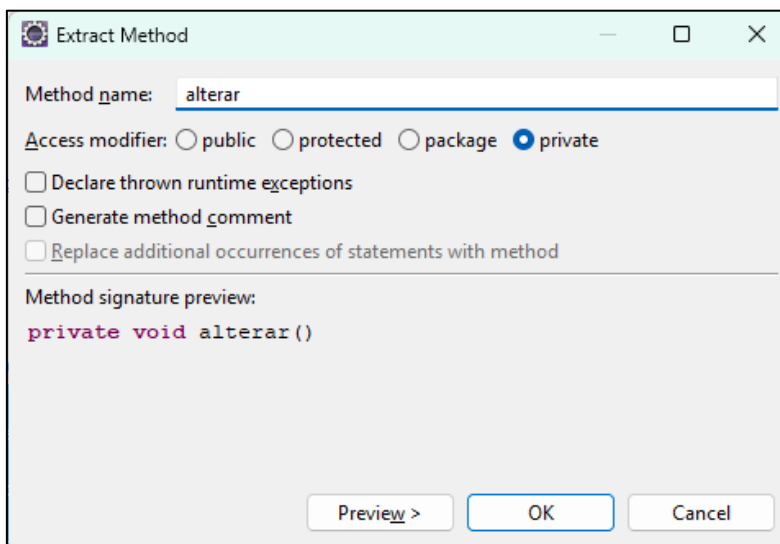
Selecione o código que está dentro do corpo do primeiro IF e aperte ALT + SHIFT + M para extrair o código para um método com o nome “cadastrar”:

```
25 private void executar() {
26
27     int opcao = 0;
28
29     do {
30         imprimirMenu();
31
32         opcao = console.readInt("Digite uma opção: ");
33
34         if (opcao == 1) {
35             System.out.println("\nCadastrar novo usuário\n");
36
37             String nome = console.readLine("Digite o nome: ");
38             long cpf = console.readLong("Digite o CPF: ");
39             LocalDate dataNascimento = console.readLocalDate("Digite a data de nascimento: ");
40
41             Usuario usuario = new Usuario();
42             usuario.setNome(nome);
43             usuario.setCpf(cpf);
44             usuario.setDataNascimento(dataNascimento);
45
46             gerenciarUsuario.salvar(usuario);
47
48             System.out.println("\nUsuário criado com sucesso\n");
49
50         } else if (opcao == 2) {
51             System.out.println("\nAlterar usuário\n");
52         }
53     } while (opcao != 0);
54 }
```



Selecione o código que está dentro do corpo do segundo IF e aperte ALT + SHIFT + M para extrair o código para um método com o nome “alterar”:

```
34         if (opcao == 1) {
35             cadastrar();
36         } else if (opcao == 2) {
37             System.out.println("\nAlterar usuário\n");
38             int idParaAlterar = console.readInt("Digite o ID para alterar: ");
39             Usuario usuario = gerenciarUsuario.findById(idParaAlterar);
40             String nome = console.readLine("Digite o nome: ");
41             long cpf = console.readLong("Digite o CPF: ");
42             LocalDate dataNascimento = console.readLocalDate("Digite a data de nascimento: ");
43             usuario.setNome(nome);
44             usuario.setCpf(cpf);
45             usuario.setDataNascimento(dataNascimento);
46             gerenciarUsuario.atualizar(usuario);
47             System.out.println("\nUsuário alterado com sucesso\n");
48         } else if (opcao == 3) {
49             System.out.println("\nListar usuários\n");
```



The image shows the 'Extract Method' dialog box in an IDE. The 'Method name' field contains the text 'alterar'. Under the 'Access modifier' section, the 'private' radio button is selected. There are three unchecked checkboxes: 'Declare thrown runtime exceptions', 'Generate method comment', and 'Replace additional occurrences of statements with method'. The 'Method signature preview' section shows the code 'private void alterar()'. At the bottom, there are three buttons: 'Preview >', 'OK', and 'Cancel'.

Method name:

Access modifier: ☐ public ☐ protected ☐ package ☒ private

☐ Declare thrown runtime exceptions

☐ Generate method comment

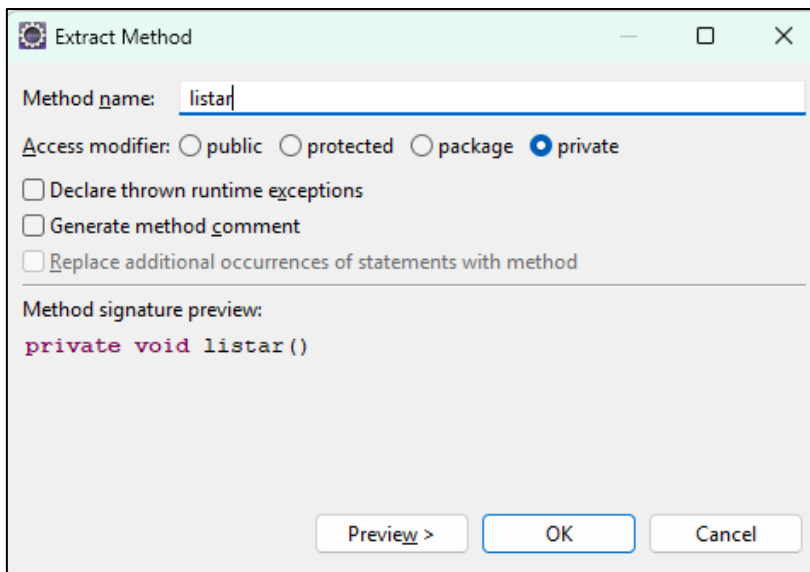
☐ Replace additional occurrences of statements with method

Method signature preview:

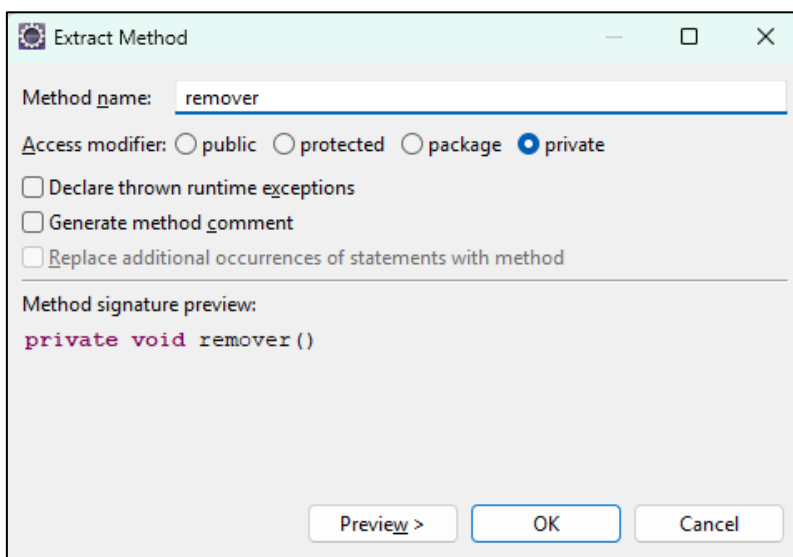
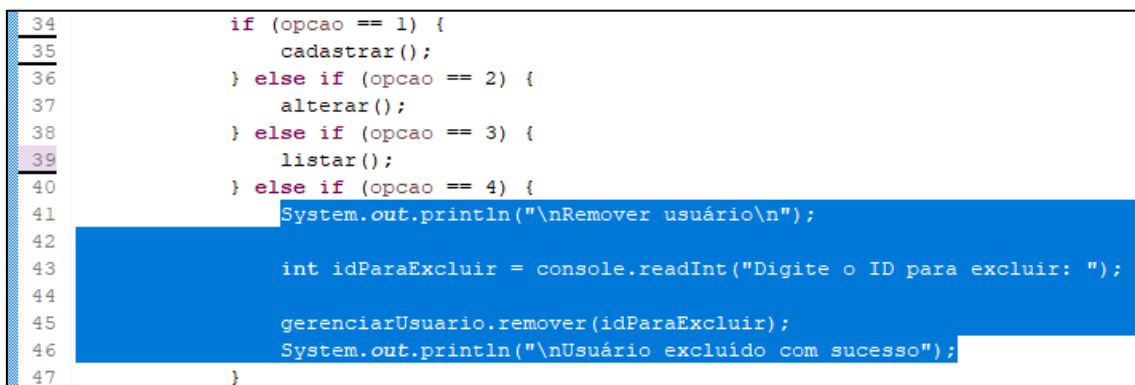
```
private void alterar()
```

Selecione o código que está dentro do corpo do terceiro IF e aperte ALT + SHIFT + M para extrair o código para um método com o nome “listar”:

```
34         if (opcao == 1) {
35             cadastrar();
36         } else if (opcao == 2) {
37             alterar();
38         } else if (opcao == 3) {
39             System.out.println("\nListar usuários\n");
40             DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
41             List<Usuario> usuarios = gerenciarUsuario.listar();
42             for (Usuario usuario : usuarios) {
43                 System.out.println("ID: " + usuario.getId());
44                 System.out.println("Nome: " + usuario.getNome());
45                 System.out.println("CPF: " + usuario.getCpf());
46                 System.out.println("Data de nascimento: " + dtf.format(usuario.getDataNascimento()));
47             }
48         } else if (opcao == 4) {
49             System.out.println("\nRemover usuário\n");
```



Selecione o código que está dentro do corpo do quarto IF e aperte ALT + SHIFT + M para extrair o código para um método com o nome “remove”:



Veja que o método executar já está bem melhor, com menos código e menos responsabilidade:

```
25 private void executar() {
26
27     int opcao = 0;
28
29     do {
30         imprimirMenu();
31
32         opcao = console.readInt("Digite uma opção: ");
33
34         if (opcao == 1) {
35             cadastrar();
36         } else if (opcao == 2) {
37             alterar();
38         } else if (opcao == 3) {
39             listar();
40         } else if (opcao == 4) {
41             remover();
42         }
43
44     } while (opcao != 9);
45
46     System.out.println("Terminando o programa, bye");
47 }
```

Ainda dá pra melhorar mais, vamos refatorar esses números mágicos nos IFs e na condição do WHILE, 1, 2, 3, 4 e 9.

## Números mágicos

Em programação, **número mágico** (ou *magic number*, em inglês) é um **número literal usado diretamente no código** sem uma explicação clara do que ele representa.

### Por que são chamados de "mágicos"?

Porque, ao olhar para o código, o valor simplesmente "aparece" do nada, sem contexto — como se fosse algo mágico. Isso dificulta a compreensão e a manutenção do código.

### Exemplo ruim (com número mágico):

```
double salarioFinal = salarioBase * 1.15;
```

Neste caso, o 1.15 é um número mágico. Quem lê o código não sabe de onde ele veio ou por que foi escolhido.

### Exemplo bom (sem número mágico):

```
double static final BONUS_ANUAL = 1.15;  
salarioFinal = salarioBase * BONUS_ANUAL;
```

Agora, o valor tem um nome explicativo, e fica muito mais fácil entender o que ele representa.

---

### Problemas dos números mágicos:

- **Dificultam a leitura** do código.
- **Complicam a manutenção** (se você precisar mudar o valor, pode esquecer de mudar em todos os lugares).
- **Aumentam a chance de bugs**, especialmente se o número for usado em vários pontos do programa.

Vamos transformar os números mágicos do nosso código em constantes com os nomes significativos, no início da classe crie os seguintes atributos:

```
private static final int CADASTRAR = 1;  
private static final int ALTERAR = 2;  
private static final int LISTAR = 3;  
private static final int REMOVER = 4;  
private static final int SAIR = 9;
```

```
11 public class ProgramaPrincipal {  
12  
13     private GerenciarUsuario gerenciarUsuario;  
14     private Console console;  
15  
16     private static final int CADASTRAR = 1;  
17     private static final int ALTERAR = 2;  
18     private static final int LISTAR = 3;  
19     private static final int REMOVER = 4;  
20     private static final int SAIR = 9;  
21
```

Agora troque os números no código pela respectiva constante:

```
32 private void executar() {
33
34     int opcao = 0;
35
36     do {
37         imprimirMenu();
38
39         opcao = console.readInt("Digite uma opção: ");
40
41         if (opcao == CADASTRAR) {
42             cadastrar();
43         } else if (opcao == ALTERAR) {
44             alterar();
45         } else if (opcao == LISTAR) {
46             listar();
47         } else if (opcao == REMOVER) {
48             remover();
49         }
50
51     } while (opcao != SAIR);
52
53     System.out.println("Terminando o programa, bye");
54 }
```

Veja como o código fica mais fácil de entender.

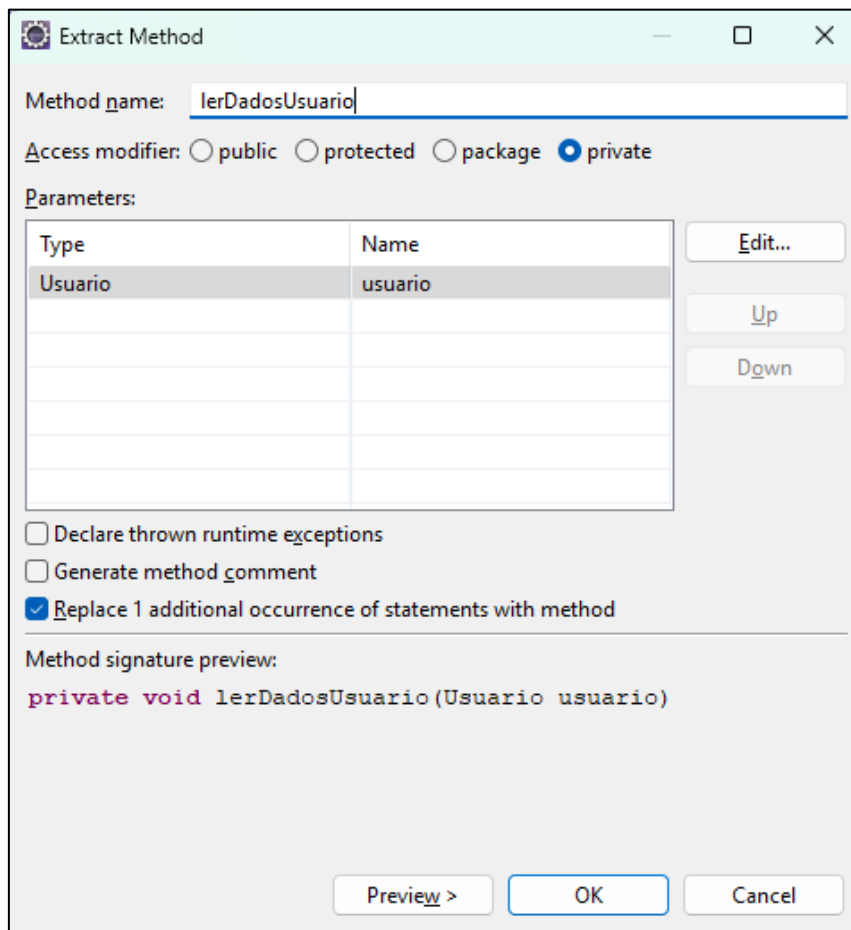
Agora procure o método “cadastrar”, vamos refatorar ele.

```
97 private void cadastrar() {
98     System.out.println("\nCadastrar novo usuário\n");
99
100     Usuario usuario = new Usuario();
101
102     String nome = console.readLine("Digite o nome: ");
103     long cpf = console.readLong("Digite o CPF: ");
104     LocalDate dataNascimento = console.readLocalDate("Digite a data de nascimento: ");
105
106     usuario.setNome(nome);
107     usuario.setCpf(cpf);
108     usuario.setDataNascimento(dataNascimento);
109
110     gerenciarUsuario.salvar(usuario);
111
112     System.out.println("\nUsuário criado com sucesso\n");
113 }
```



Selecione essa parte do código e aperte ALT + SHIFT + M para extrair esse bloco de código para um método chamado “lerDadosUsuario”:

```
97 private void cadastrar() {
98     System.out.println("\nCadastrar novo usuário\n");
99
100     Usuario usuario = new Usuario();
101
102     String nome = console.readLine("Digite o nome: ");
103     long cpf = console.readLong("Digite o CPF: ");
104     LocalDate dataNascimento = console.readLocalDate("Digite a data de nascimento: ");
105
106     usuario.setNome(nome);
107     usuario.setCpf(cpf);
108     usuario.setDataNascimento(dataNascimento);
109
110     gerenciarUsuario.salvar(usuario);
111
112     System.out.println("\nUsuário criado com sucesso\n");
113 }
```



The image shows the 'Extract Method' dialog box in an IDE. The 'Method name' field contains 'lerDadosUsuario'. The 'Access modifier' section has 'private' selected. The 'Parameters' table has one row with 'Usuario' as the type and 'usuario' as the name. The 'Replace 1 additional occurrence of statements with method' checkbox is checked. The 'Method signature preview' shows 'private void lerDadosUsuario(Usuario usuario)'. At the bottom are 'Preview >', 'OK', and 'Cancel' buttons.

Method name:

Access modifier: ☐ public ☐ protected ☐ package ☒ private

Parameters:

| Type    | Name    |
|---------|---------|
| Usuario | usuario |
|         |         |
|         |         |
|         |         |
|         |         |
|         |         |

☐ Declare thrown runtime exceptions  
☐ Generate method comment  
☒ Replace 1 additional occurrence of statements with method

Method signature preview:  
`private void lerDadosUsuario(Usuario usuario)`

O Eclipse irá procurar todas as ocorrências desse bloco de código e irá substituir o bloco pela chamada de método correspondente, repare que ele refatorou o método “**cadast**rar” e “**alterar**”:

```
77 private void alterar() {
78     System.out.println("\nAlterar usuário\n");
79
80     int idParaAlterar = console.readInt("Digite o ID para alterar: ");
81
82     Usuario usuario = gerenciarUsuario.findById(idParaAlterar);
83
84     lerDadosUsuario(usuario);
85
86     gerenciarUsuario.atualizar(usuario);
87
88     System.out.println("\nUsuário alterado com sucesso\n");
89 }
90
91 private void cadastrar() {
92     System.out.println("\nCadastrar novo usuário\n");
93
94     Usuario usuario = new Usuario();
95
96     lerDadosUsuario(usuario);
97
98     gerenciarUsuario.salvar(usuario);
99
100    System.out.println("\nUsuário criado com sucesso\n");
101 }
```

Por fim, vamos refatorar o método “listar”. Primeiro modifique o código para que ele fique igual ao código abaixo (inverte a posição das linhas 68 para a 67:

ANTES:

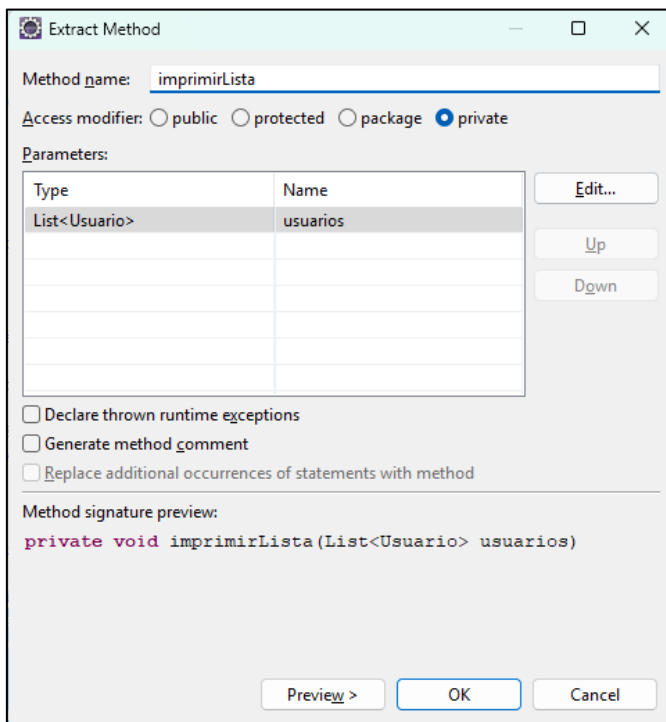
```
65 private void listar() {
66     System.out.println("\nListar usuários\n");
67     DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
68     List<Usuario> usuarios = gerenciarUsuario.listar();
69     for (Usuario usuario : usuarios) {
70         System.out.println("\nID: " + usuario.getId());
71         System.out.println("Nome: " + usuario.getNome());
72         System.out.println("CPF: " + usuario.getCpf());
73         System.out.println("Data de nascimento: " + dtf.format(usuario.getDataNascimento()));
74     }
75 }
```

DEPOIS:

```
65 private void listar() {
66     System.out.println("\nListar usuários\n");
67
68     List<Usuario> usuarios = gerenciarUsuario.listar();
69     DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
70     for (Usuario usuario : usuarios) {
71         System.out.println("\nID: " + usuario.getId());
72         System.out.println("Nome: " + usuario.getNome());
73         System.out.println("CPF: " + usuario.getCpf());
74         System.out.println("Data de nascimento: " + dtf.format(usuario.getDataNascimento()));
75     }
76 }
```

Agora, selecione o bloco abaixo e aperte ALT + SHIFT + M para extrair o bloco de código para o método “imprimirLista”:

```
65 private void listar() {  
66     System.out.println("\nListar usuários\n");  
67  
68     List<Usuario> usuarios = gerenciarUsuario.listar();  
69     DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");  
70     for (Usuario usuario : usuarios) {  
71         System.out.println("\nID: " + usuario.getId());  
72         System.out.println("Nome: " + usuario.getNome());  
73         System.out.println("CPF: " + usuario.getCpf());  
74         System.out.println("Data de nascimento: " + dtf.format(usuario.getDataNascimento()));  
75     }  
76 }
```



The image shows the 'Extract Method' dialog box in an IDE. The 'Method name' field contains 'imprimirLista'. The 'Access modifier' section has radio buttons for 'public', 'protected', 'package', and 'private', with 'private' selected. The 'Parameters' section contains a table with one row: 'List<Usuario>' and 'usuarios'. To the right of the table are buttons for 'Edit...', 'Up', and 'Down'. Below the table are three checkboxes: 'Declare thrown runtime exceptions', 'Generate method comment', and 'Replace additional occurrences of statements with method', all of which are unchecked. The 'Method signature preview' section shows the code: 'private void imprimirLista(List<Usuario> usuarios)'. At the bottom are buttons for 'Preview >', 'OK', and 'Cancel'.

Method name:

Access modifier: ☐ public ☐ protected ☐ package ☒ private

Parameters:

| Type          | Name     |
|---------------|----------|
| List<Usuario> | usuarios |
|               |          |
|               |          |
|               |          |
|               |          |

☐ Declare thrown runtime exceptions  
☐ Generate method comment  
☐ Replace additional occurrences of statements with method

Method signature preview:  
`private void imprimirLista(List<Usuario> usuarios)`

Pronto, terminamos a refatoração do código, temos o programa usando orientação a objetos, métodos curtos, cada um com sua responsabilidade e o código bem legível.

## Código final completo da classe “ProgramaPrincipal”

```
package main;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;

import console.Console;
import usuario.Usuario;
import usuario.gerenciar.GerenciarUsuario;

public class ProgramaPrincipal {

    private GerenciarUsuario gerenciarUsuario;
    private Console console;

    private static final int CADASTRAR = 1;
    private static final int ALTERAR = 2;
    private static final int LISTAR = 3;
    private static final int REMOVER = 4;
    private static final int SAIR = 9;

    public ProgramaPrincipal() {
        gerenciarUsuario = new GerenciarUsuario();
        console = new Console();
    }

    public static void main(String[] args) {
        new ProgramaPrincipal().executar();
    }

    private void executar() {

        int opcao = 0;

        do {
            imprimirMenu();

            opcao = console.readInt("Digite uma opção: ");

            if (opcao == CADASTRAR) {
                cadastrar();
            } else if (opcao == ALTERAR) {
                alterar();
            } else if (opcao == LISTAR) {
                listar();
            } else if (opcao == REMOVER) {
                remover();
            }

        } while (opcao != SAIR);

        System.out.println("Terminando o programa, bye");
    }

    private void remover() {
        System.out.println("\nRemover usuário\n");

        int idParaExcluir = console.readInt("Digite o ID para excluir: ");

        gerenciarUsuario.remover(idParaExcluir);
    }
}
```

```

        System.out.println("\nUsuário excluído com sucesso");
    }

    private void listar() {
        System.out.println("\nListar usuários\n");

        List<Usuario> usuarios = gerenciarUsuario.listar();
        imprimirLista(usuarios);
    }

    private void imprimirLista(List<Usuario> usuarios) {
        DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
        for (Usuario usuario : usuarios) {
            System.out.println("\nID: " + usuario.getId());
            System.out.println("Nome: " + usuario.getNome());
            System.out.println("CPF: " + usuario.getCpf());
            System.out.println("Data de nascimento: " +
dtf.format(usuario.getDataNascimento()));
        }
    }

    private void alterar() {
        System.out.println("\nAlterar usuário\n");

        int idParaAlterar = console.readInt("Digite o ID para alterar: ");

        Usuario usuario = gerenciarUsuario.findById(idParaAlterar);

        lerDadosUsuario(usuario);

        gerenciarUsuario.atualizar(usuario);

        System.out.println("\nUsuário alterado com sucesso\n");
    }

    private void cadastrar() {
        System.out.println("\nCadastrar novo usuário\n");

        Usuario usuario = new Usuario();

        lerDadosUsuario(usuario);

        gerenciarUsuario.salvar(usuario);

        System.out.println("\nUsuário criado com sucesso\n");
    }

    private void lerDadosUsuario(Usuario usuario) {
        String nome = console.readLine("Digite o nome: ");
        long cpf = console.readLong("Digite o CPF: ");
        LocalDate dataNascimento = console.readLocalDate("Digite a data de
nascimento: ");

        usuario.setNome(nome);
        usuario.setCpf(cpf);
        usuario.setDataNascimento(dataNascimento);
    }

    private void imprimirMenu() {
        System.out.println("");
        System.out.println("--- SUPER CRUD ---");
        System.out.println("");
        System.out.println("1 - Cadastrar usuário");
    }

```

```
        System.out.println("2 - Alterar usuário");  
        System.out.println("3 - Listar usuários");  
        System.out.println("4 - Remover usuário");  
        System.out.println("9 - Sair");  
    }  
}
```

## GitHub

Caso você queira o código fonte desse documento, visite o GitHub.

[https://github.com/paulorvj/crud\\_memoria](https://github.com/paulorvj/crud_memoria)

Você pode clonar o repositório ou fazer o download do código em um arquivo zip.

# Testes unitários

## Introdução

### O que são testes unitários?

**Teste unitário** é um tipo de teste automatizado que verifica se uma **unidade do código** (geralmente um método) funciona corretamente de forma **isolada**.

 A ideia é: “Dado um certo valor de entrada, o método retorna o resultado esperado?”

---

### Por que usar testes unitários?

- **Garante que seu código funciona como esperado**
- **Facilita manutenção:** você sabe se quebrou algo quando alterar o código
- **Evita bugs escondidos**
- **Ajuda no design:** escrever código testável normalmente leva a um design melhor e mais modular

---

### Ferramentas mais comuns em Java

- **JUnit (mais usado)** – Atualmente na versão JUnit 5
- Mockito – Para simular (mockar) dependências em testes unitários
- AssertJ / Hamcrest – Para melhorar as mensagens de asserção

---

### Exemplo básico com JUnit 5

Suponha que temos uma classe simples:

```
public class Calculadora {  
    public int somar(int a, int b) {  
        return a + b;  
    }  
}
```

Agora vamos escrever um teste unitário para esse método:

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.assertEquals;  
  
public class CalculadoraTest {  
  
    @Test  
    public void testSomar() {  
        Calculadora calc = new Calculadora();  
        int resultado = calc.somar(2, 3);  
        assertEquals(5, resultado); // Verifica se o resultado  
esperado é 5  
    }  
}
```

### Explicando:

- @Test: indica que o método é um teste.
- assertEquals(expected, actual): verifica se o resultado é o esperado.
- Se a asserção falhar, o teste falha.

---

### Como executar?

Se você estiver usando:

- **IntelliJ ou Eclipse:** basta clicar no botão de “play” ao lado do método ou classe de teste.
- **Maven:** mvn test
- **Gradle:** ./gradlew test

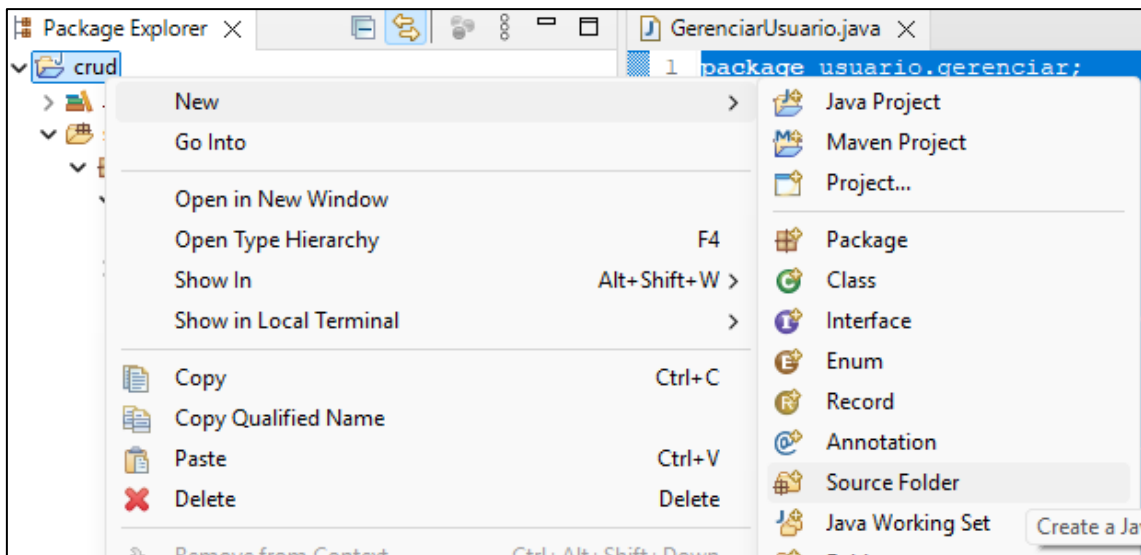
---

### Dicas rápidas

- Um teste **não deve depender** de outro.
- Um teste não deve depender de uma ordem de execução para que funcione.
- Mantenha os testes **curtos e simples**.
- Use nomes descritivos para os métodos de teste, tipo deveSomarCorretamente().
- Faça testes cobrindo **casos normais, casos limite e valores inválidos**.

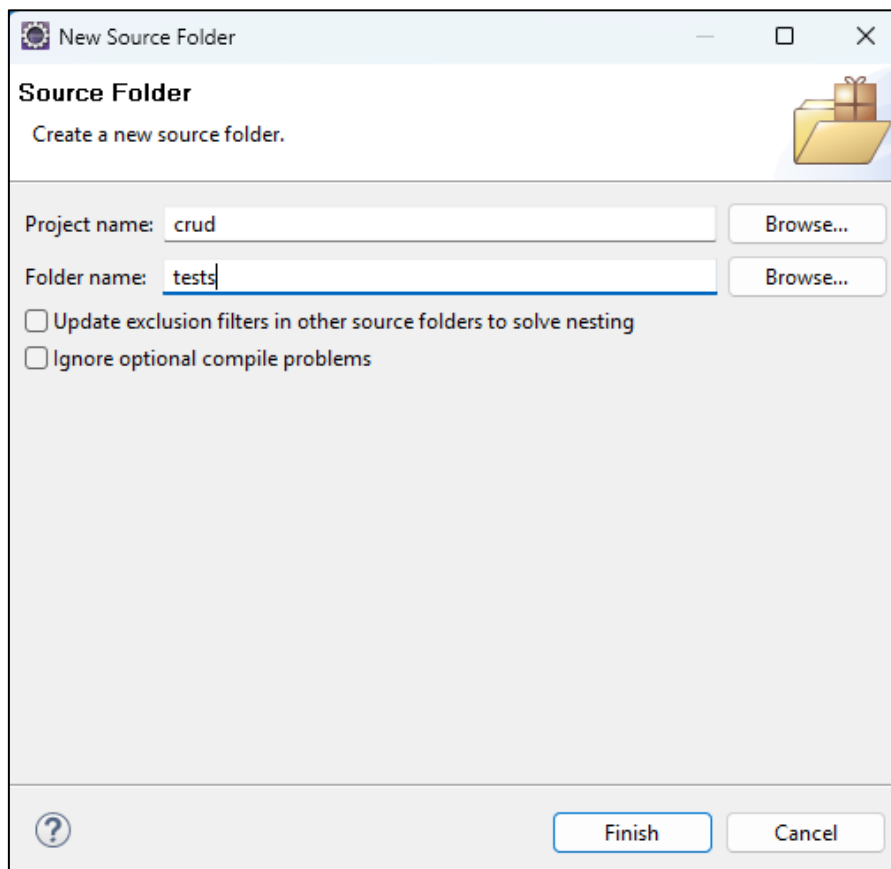
## Implementação

Vamos iniciar a implementação dos testes unitários com JUnit. Para isso crie uma nova pasta “source” no projeto, clique com o botão direito no projeto e vá no menu “New – Source Folder”:

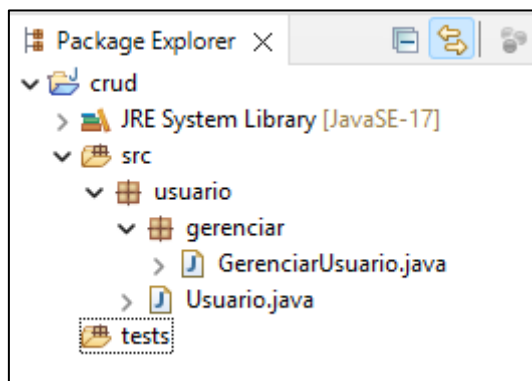




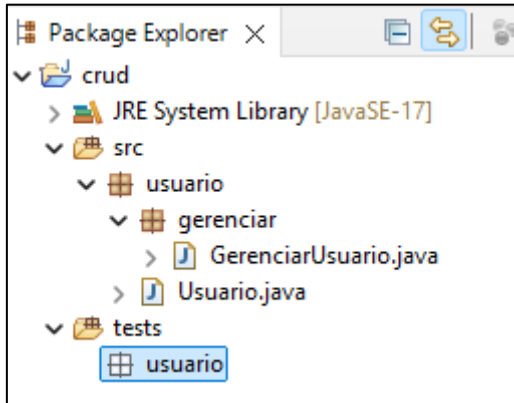
Coloque como nome da pasta “tests” e clique em “Finish”:



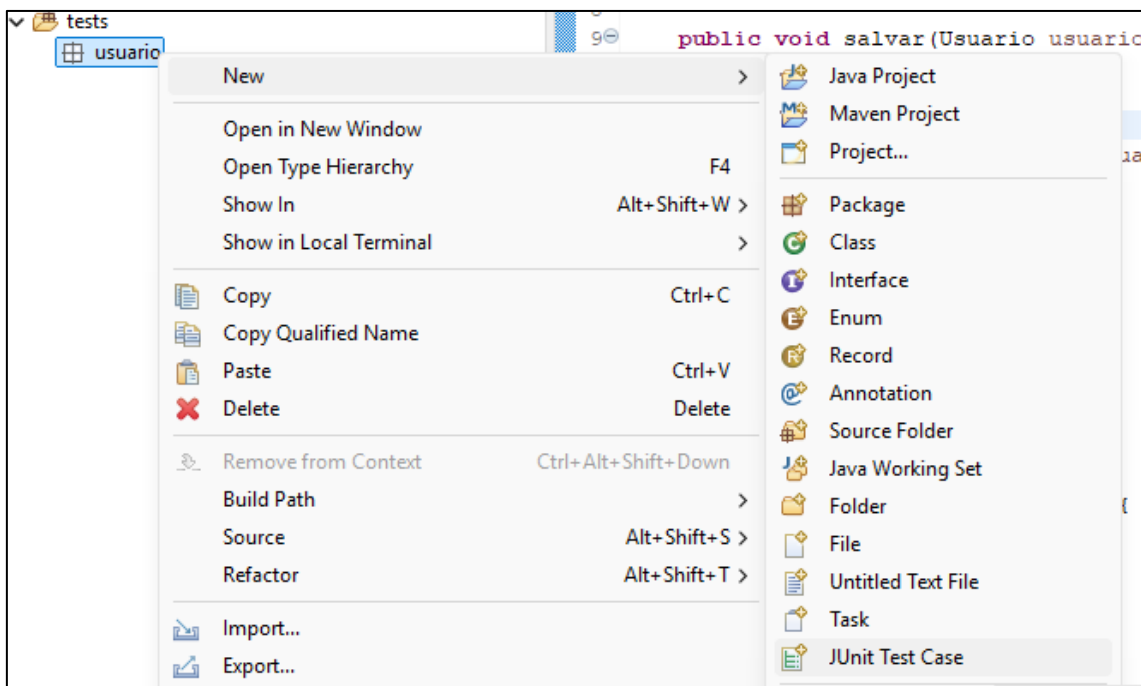
A estrutura do projeto ficará assim:



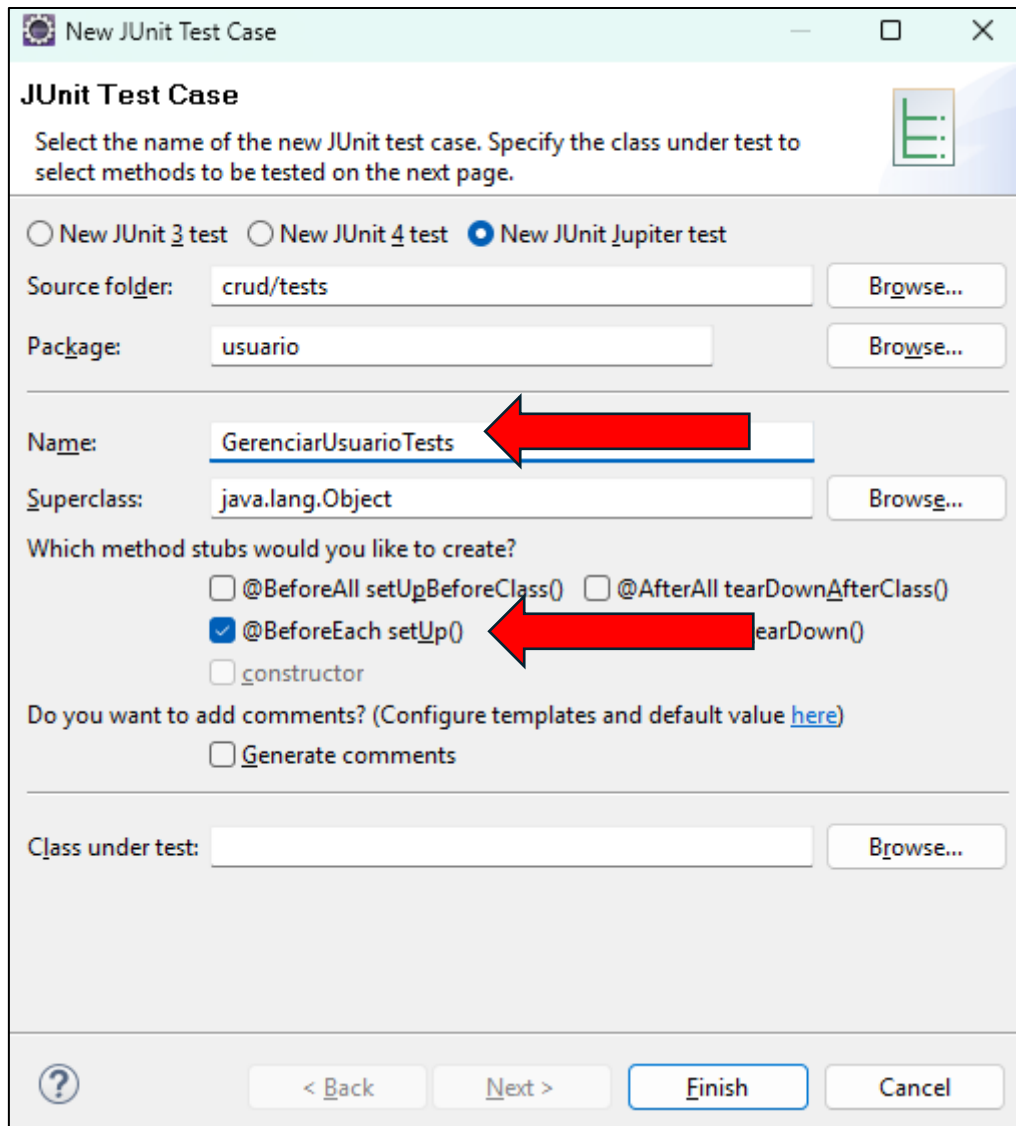
Crie um pacote chamado “usuario” na “source folder” – “tests”:



Vamos testar a classe “GerenciarUsuario”, para isso crie uma classe de teste chamada “GerenciarUsuarioTests”. Clique com o botão direito do mouse no pacote “usuario” e vá em “new – JUnit Test Case”:



Coloque o nome da classe “GerenciarUsuarioTests” e marque a caixa: “@BeforeEach setUp ()” e clique em “Finish”:



**New JUnit Test Case**

Select the name of the new JUnit test case. Specify the class under test to select methods to be tested on the next page.

☐ New JUnit 3 test ☐ New JUnit 4 test ☒ New JUnit Jupiter test

Source folder: crud/tests Browse...

Package: usuario Browse...

Name: GerenciarUsuarioTests

Superclass: java.lang.Object Browse...

Which method stubs would you like to create?

☐ @BeforeAll setUpBeforeClass() ☐ @AfterAll tearDownAfterClass()

☒ @BeforeEach setUp() ☐ tearDown()

☐ constructor

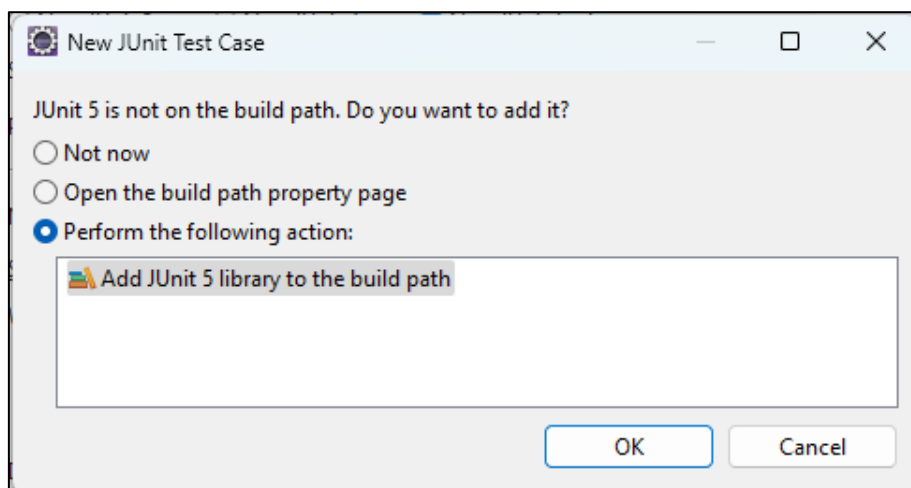
Do you want to add comments? (Configure templates and default value [here](#))

☐ Generate comments

Class under test: Browse...

[?](#) < Back Next > Finish Cancel

O Eclipse irá perguntar se você quer adicionar a lib JUnit ao projeto, clique OK:



**New JUnit Test Case**

JUnit 5 is not on the build path. Do you want to add it?

☐ Not now

☐ Open the build path property page

☒ Perform the following action:

Add JUnit 5 library to the build path

OK Cancel

O código estará dessa maneira:

```
package usuario;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class GerenciarUsuarioTests {

    @BeforeEach
    void setUp() throws Exception {
    }

    @Test
    void test() {
        fail("Not yet implemented");
    }

}
```

## Cenários

Em testes unitários, **cenários** são **situações específicas** que você quer simular/testar para garantir que o comportamento do seu código está correto, independentemente das condições.

---

### O que exatamente são cenários?

Um **cenário de teste** é como uma **história curta com contexto, ação e expectativa**.

Exemplo:

- **Cenário:** "Salvar um usuário novo"
  - **Contexto:** o sistema está vazio
  - **Ação:** salvar um novo usuário
  - **Expectativa:** o usuário é salvo com ID 1

---

### Tipos comuns de cenários em testes unitários:

#### 1. **Cenários positivos (felizes)**

Testam se o código funciona como esperado em situações normais.

- Ex: salvar um usuário válido
- Ex: listar usuários quando há registros

## 2. Cenários negativos (de erro ou exceção)

Testam se o código **lida bem com situações inválidas** ou inesperadas.

- Ex: tentar remover um usuário que não existe
- Ex: salvar um usuário com nome nulo (se for inválido)

## 3. Cenários de limite (edge cases)

Verificam o comportamento em condições extremas.

- Ex: listar usuários quando não há nenhum
- Ex: salvar muitos usuários (testar desempenho ou limites)

---

### Exemplo prático:

Para o método `remover(int id)` da sua classe, podemos ter os seguintes cenários:

| Tipo     | Cenário                               | Esperado                       |
|----------|---------------------------------------|--------------------------------|
| Positivo | Remover usuário que existe            | Usuário é removido com sucesso |
| Negativo | Remover usuário com ID que não existe | Nada acontece, sem exceção     |
| Limite   | Remover usuário com ID 0 ou negativo  | Deve ser tratado corretamente  |

### Por que pensar em cenários?

Porque eles ajudam você a **cobrir mais possibilidades** e escrever testes mais completos. Cada cenário representa uma **garantia de que seu código se comporta de forma segura e previsível**.

Vamos criar os seguintes testes:

- `testCadastrar`
- `testAlterar`
- `testListar`
- `testRemover`

Estes são apenas alguns testes básicos, não cobrem todos os cenários, o ideal é que os testes cubram o máximo possível de cenários e assim minimizar o risco de o código possuir bugs. Como o objetivo aqui é apenas ensinar como os testes funcionam, vamos apenas fazer os testes básicos acima.

Geralmente fazemos os testes antes de implementar o código do `GerenciarUsuario`, assim conseguimos ter uma ideia melhor de como o `Gerenciar` deverá funcionar e o que deve possuir, fazendo com que os testes inicialmente falhem e conforme vamos implementando o código os testes passam a obter sucesso na execução.

Coloque os métodos acima na classe de testes, o código deverá ficar assim:

```
package usuario;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

class GerenciarUsuarioTests {

    @BeforeEach
    void setUp() throws Exception {
    }

    @Test
    void testCadastrar() {
        fail("Not yet implemented");
    }

    @Test
    void testAlterar() {
        fail("Not yet implemented");
    }

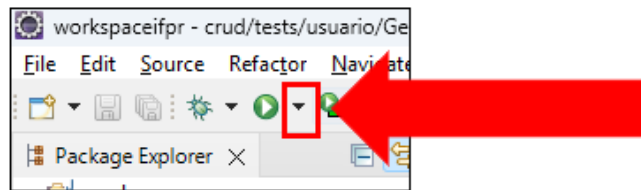
    @Test
    void testListar() {
        fail("Not yet implemented");
    }

    @Test
    void testRemover() {
        fail("Not yet implemented");
    }

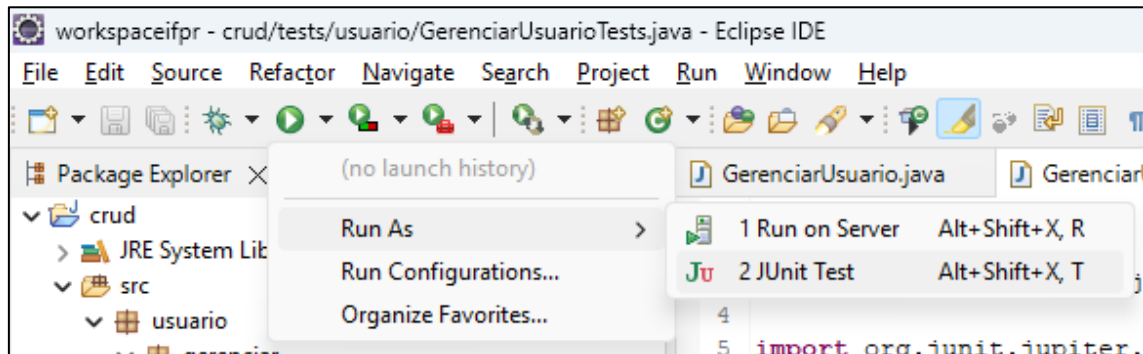
}
```

Vamos executar os testes e ver todos eles falharem 😊.

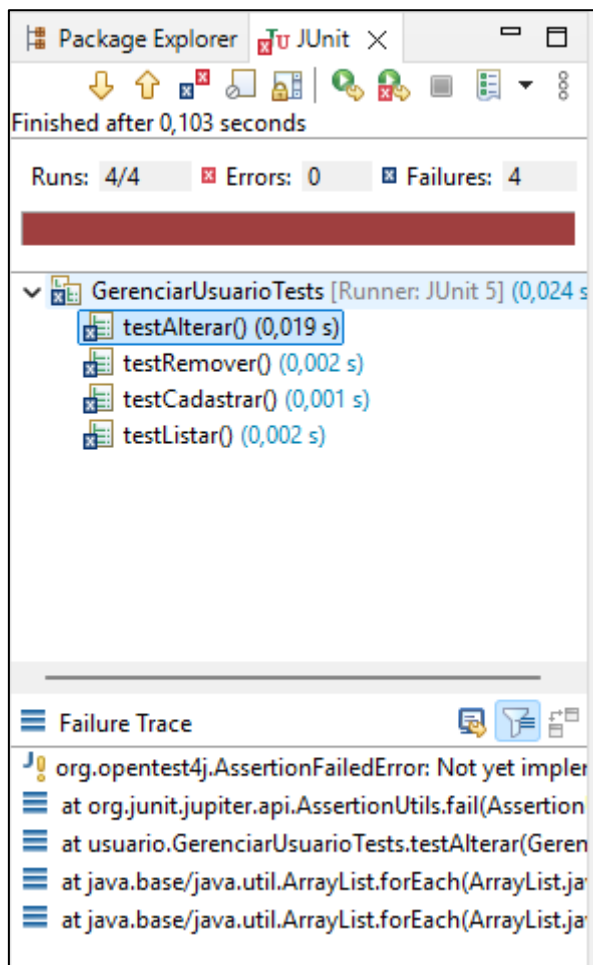
Clique na seta do botão para executar código:



Vá na opção “Run – Run as JUnit test”:



O Eclipse irá abrir a aba JUnit e mostrar o resultado da execução dos testes:



## @BeforeEach

### O que é @BeforeEach?

É uma **\*\*anotação** que marca um método para ser executado **\*\*antes de **cada** método de teste.**

### Para que serve?

Ela é usada para **preparar o ambiente de teste**. Por exemplo:

- Criar objetos que serão usados em cada teste
- Inicializar variáveis
- Resetar estados para garantir que os testes não influenciem uns aos outros

### Exemplo prático:

Antes de implementar os testes, devemos preparar o ambiente. Os testes precisaram da classe “GerenciarUsuario”, então vamos definir um atributo na classe de teste:

```
private static GerenciarUsuario gerenciarUsuario;
```

E no método setUpBeforeClass() vamos inicializar o atributo:

```
@BeforeEach
void setUp() throws Exception {
    gerenciarUsuario = new GerenciarUsuario();
}
```

Nesse caso, **antes de cada teste**, será criado um novo objeto GerenciarUsuario, garantindo que os testes sejam **isolados e independentes**.

### Diferença entre @BeforeEach e @BeforeAll

| Anotação    | Executa...                 | Requer <code>static</code> ? |
|-------------|----------------------------|------------------------------|
| @BeforeEach | Antes de cada teste        | ✗ Não precisa                |
| @BeforeAll  | Uma vez só, antes de todos | ✓ Precisa                    |

Use @BeforeEach quando **cada teste precisa de uma nova preparação**. Use @BeforeAll quando quiser fazer uma **única configuração compartilhada** por todos os testes (ex: abrir conexão com banco).



## Métodos assert

Os métodos assert são o **coração dos testes unitários** — eles verificam se o resultado do seu código **bate com o esperado**. Se a asserção falhar, o teste falha.

Aqui estão os **principais métodos de asserção** do JUnit 5 (da classe `org.junit.jupiter.api.Assertions`):

---

### ✅ 1. `assertEquals(expected, actual)`

Verifica se dois valores são iguais.

```
assertEquals(5, resultado);
```

---

### ❌ 2. `assertNotEquals(unexpected, actual)`

Verifica se dois valores **não** são iguais.

```
assertNotEquals(0, usuario.getId());
```

---

### 🤖 3. `assertTrue(condition)`

Verifica se a condição é **verdadeira**.

```
assertTrue(lista.contains(usuario));
```

---

### 🚫 4. `assertFalse(condition)`

Verifica se a condição é **falsa**.

```
assertFalse(lista.isEmpty());
```

---

### 💡 5. `assertNull(value)`

Verifica se o valor é **nulo**.

```
assertNull(usuarioNaoEncontrado);
```

---

### 🔍 6. `assertNotNull(value)`

Verifica se o valor **não** é nulo.

```
assertNotNull(usuario.getNome());
```

---

## 7. `assertThrows(Exception.class, () -> { ... })`

Verifica se uma exceção esperada é lançada.

```
assertThrows(IllegalArgumentException.class, () -> {  
    usuarioService.salvar(null);  
});
```

---

## 8. `assertAll(...)`

Permite agrupar **múltiplas asserções** em um único teste.

```
assertAll(  
    () -> assertEquals("João", usuario.getNome()),  
    () -> assertEquals(1, usuario.getId())  
);
```

---

## Extra: Personalizar mensagem de erro

Você pode passar uma **mensagem opcional** como terceiro argumento:

```
assertEquals(5, resultado, "A soma deveria ser 5");
```

## Teste do cadastro

Vamos começar com o cadastro.

O cadastro deve funcionar da seguinte maneira:

- Criar um objeto da classe `Usuario` e definir alguns valores para nome, cpf e data de nascimento. Não vamos definir um id, pois, esperamos que o id seja gerado pelo banco de dados.
- Chamar o “gerenciar” e pedir para ele salvar o novo objeto.
- em seguida vamos usar o método `findById` para buscar esse usuário que acabamos de criar, esperamos que o id dele seja 1.
- Com o objeto que foi retornado pelo `findById`, vamos verificar se as informações que definimos na criação do usuário são as mesmas que vieram no objeto recuperado do banco de dados.
- Se as informações baterem, o teste deve ser considerado sucesso.

Primeiro apague a linha “fail(“Not yet implemented”);” do método `testCadastrar`.

Crie um objeto da classe `Usuario` e defina alguns valores para os atributos nome, cpf e data de nascimento:

```
Usuario usuario = new Usuario();
usuario.setNome("Chuck Norris");
usuario.setCpf(12312312311L);
usuario.setDataNascimento(LocalDate.of(2012, 12, 12));
```

Use o objeto “gerenciarUsuario” e chame o método “salvar” passando como parâmetro o objeto usuário:

```
gerenciarUsuario.salvar(usuario);
```

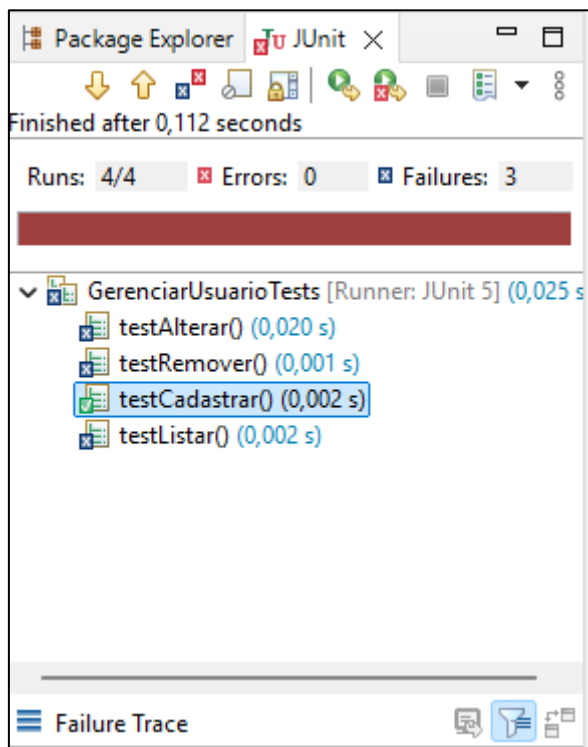
Agora vamos buscar o objeto salvo no banco usando o método `findById`:

```
Usuario usuarioBD = gerenciarUsuario.findById(1);
```

O último passo é verificar as informações do objeto que veio do banco de dados com o objeto que criamos:

```
assertEquals("Chuck Norris", usuarioBD.getNome());
assertEquals(12312312311L, usuarioBD.getCpf());
assertEquals(LocalDate.of(2012, 12, 12), usuarioBD.getDataNascimento());
```

Execute novamente os testes, veja que nosso testCadastrar ficou verde:



Código do método “testCadastrar()”:

```
@Test
void testCadastrar() {
    Usuario usuario = new Usuario();
    usuario.setNome("Chuck Norris");
    usuario.setCpf(12312312311L);
    usuario.setDataNascimento(LocalDate.of(2012, 12, 12));

    gerenciarUsuario.salvar(usuario);

    Usuario usuarioBD = gerenciarUsuario.findById(1);

    assertEquals("Chuck Norris", usuarioBD.getNome());
    assertEquals(12312312311L, usuarioBD.getCpf());
    assertEquals(LocalDate.of(2012, 12, 12), usuarioBD.getDataNasciment
o());
}
```

## Teste de alteração de usuário

A alteração deve funcionar da seguinte maneira:

- 1 – Criar um objeto da classe `Usuario` e definir alguns valores para nome, cpf e data de nascimento.
- 2 – Chamar o “`gerenciarUsuario`” e pedir para ele salvar o novo objeto.
- 3 – Chamar o método “`findById()`” para buscar esse usuário que acabamos de criar, esperamos que o id dele seja 1.
- 4 – Alterar as informações do objeto que foi retornado do banco de dados.
- 5 – Chamar o “`gerenciarUsuario`” e pedir para atualizar o objeto no banco de dados.
- 6 – Chamar o método “`findById()`” para buscar novamente o usuário que acabamos de alterar.
- 7 – Comparamos os dados do usuário atualizado com o objeto alterado.
- 8 – Se as informações baterem, o teste deve ser considerado sucesso.

Código do teste:

```
@Test
void testAlterar() {
    Usuario usuario = new Usuario();
    usuario.setNome("Chuck Norris");
    usuario.setCpf(12312312311L);
    usuario.setDataNascimento(LocalDate.of(2012, 12, 12));

    gerenciarUsuario.salvar(usuario);

    Usuario usuarioBd = gerenciarUsuario.findById(1);

    usuarioBd.setNome("Novo nome");
    usuarioBd.setCpf(22222222222L);
    usuarioBd.setDataNascimento(LocalDate.of(2010, 10, 10));

    gerenciarUsuario.atualizar(usuarioBd);

    Usuario usuarioAtualizado = gerenciarUsuario.findById(1);

    assertEquals(usuarioBd.getNome(), usuarioAtualizado.getNome());
    assertEquals(usuarioBd.getCpf(), usuarioAtualizado.getCpf());
    assertEquals(usuarioBd.getDataNascimento(), usuarioAtualizado.getDataNascimento());
}
```

## Teste listar usuários

O listar deve funcionar da seguinte maneira:

- 1 – Criar dois objetos objeto da classe Usuario e definir alguns valores para nome, cpf e data de nascimento.
- 2 – Chamar o “gerenciarUsuario” e pedir para ele salvar os dois objetos.
- 3 – Chamar o método listar e verificar se o tamanho da lista é 2.
- 4 – Se o tamanho da lista bater, o teste deve ser considerado sucesso.

Código do teste:

```
@Test
void testListar() {
    Usuario usuarioUm = new Usuario();
    usuarioUm.setNome("Chuck Norris");
    usuarioUm.setCpf(12312312311L);
    usuarioUm.setDataNascimento(LocalDate.of(2012, 12, 12));

    Usuario usuarioDois = new Usuario();
    usuarioDois.setNome("Novo nome");
    usuarioDois.setCpf(22222222222L);
    usuarioDois.setDataNascimento(LocalDate.of(2010, 10, 10));

    gerenciarUsuario.salvar(usuarioUm);
    gerenciarUsuario.salvar(usuarioDois);

    List<Usuario> usuarios = gerenciarUsuario.listar();
    assertEquals(2, usuarios.size());
}
```

## Teste remover usuário

O remover deve funcionar da seguinte maneira:

- 1 – Criar um objetos objeto da classe Usuario e definir alguns valores para nome, cpf e data de nascimento.
- 2 – Chamar o “gerenciarUsuario” e pedir para ele salvar o objeto.
- 3 – Chamar o método listar e verificar se o tamanho da lista é 1.
- 4 – Remover o objeto criado, ID 1.
- 5 – Buscar o usuário pelo ID 1, como o objeto deve ter sido removido, o findById deve retornar “null”.
- 6 – Verificar se o objeto retornado é realmente nulo.
- 4 – Se o objeto for nulo, o teste deve ser considerado sucesso.

Código do teste:

```
@Test
void testRemover() {
    Usuario usuarioUm = new Usuario();
    usuarioUm.setNome("Chuck Norris");
    usuarioUm.setCpf(12312312311L);
    usuarioUm.setDataNascimento(LocalDate.of(2012, 12, 12));

    gerenciarUsuario.salvar(usuarioUm);

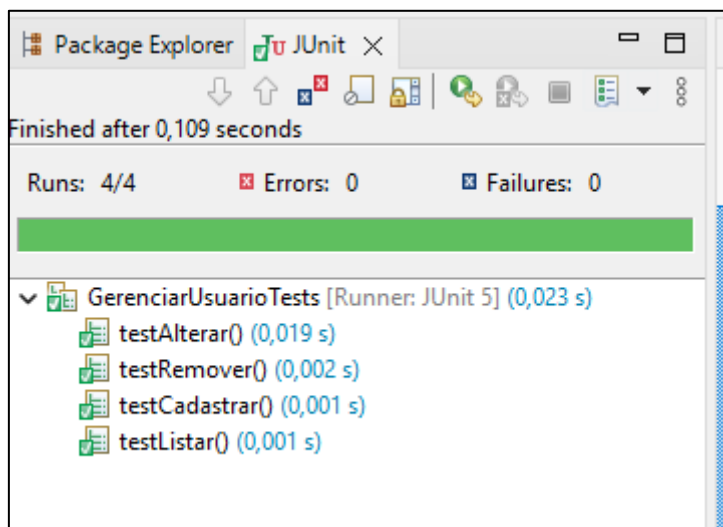
    List<Usuario> usuarios = gerenciarUsuario.listar();
    assertEquals(1, usuarios.size());

    gerenciarUsuario.remover(1);

    Usuario usuario = gerenciarUsuario.findById(1);

    assertNull(usuario);
}
```

Execute novamente os teste, veja que todos os testes passam com sucesso:



Concluindo, agora temos alguns testes do nosso projeto, não cobrem todos os cenários e nosso programa não está livre de bugs, mas com isso temos noção que o CRUD básico está funcionando. Caso façamos alguma alteração no código, temos alguns testes para garantir que pelo menos essa parte básica continue funcionando.



## Classe Console

Crie um pacote “console” e dentro a classe “Console”, copie e cole o código a seguir:

```
package console;

import java.math.BigDecimal;
import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.Date;

public class Console {

    private SimpleDateFormat dateFormatter;

    public Console() {
        dateFormatter = new SimpleDateFormat();
    }

    public void printMessage(String message) {
        if (!message.equals("")) {
            System.out.println(message.trim() + " ");
            System.out.flush();
        }
    }

    public void printDate(Date date) {
        if (date != null) {
            System.out.println(dateFormatter.format(date));
            System.out.flush();
        }
    }

    public void printDate(Date date, String format) {
        if (date != null) {
            dateFormatter.applyPattern(format);
            System.out.println(dateFormatter.format(date));
            System.out.flush();
        }
    }

    public String readLine() {
        int tCh;
        String tResult = "";
        boolean tDone = false;

        while (!tDone) {
            try {
                tCh = System.in.read();
                if (tCh < 0 || (char) tCh == '\n')
                    tDone = true;
                else if ((char) tCh != '\r') // weird--it used to do \r\n
                    // translation
                    tResult = tResult + (char) tCh;
            }
        }
    }
}
```

```

        } catch (java.io.IOException tExcept) {
            tDone = true;
        }
    }
    return tResult;
}

public BigDecimal readBigDecimal(String message) {
    printMessage(message);
    return this.readBigDecimal();
}

public BigDecimal readBigDecimal() {
    String valor = this.readString();
    BigDecimal bd = new BigDecimal(valor);
    return bd;
}

public String readLine(String message) {
    printMessage(message);
    return readLine();
}

public String readString(String message) {
    return readLine(message);
}

public String readString() {
    return readLine();
}

public char readChar() {
    char tResult = '\0';
    int tCh;
    boolean tDone = false;
    boolean tRead = false;

    while (!tDone) {
        try {
            tCh = System.in.read();
            if (tCh < 0 || (char) tCh == '\n')
                tDone = true;
            else if (!tRead && (char) tCh != '\r') {
                tResult = (char) tCh;
                tRead = true;
            }
        } catch (java.io.IOException tExcept) {
            tDone = true;
        }
    }
    return tResult;
}

public char readChar(String message) {
    printMessage(message);
    return readChar();
}

public double readDouble() {

```

```

        return readDouble("");
    }

    public double readDouble(String message) {
        String tLinha;

        while (true) {
            try {
                printMessage(message);
                tLinha = readLine().trim();
                if (tLinha.equals(""))
                    return 0.0;
                return Double.parseDouble(tLinha);
            } catch (NumberFormatException e) {
                System.out.println("Not an double. Please try again!");
            }
        }
    }

    public float readFloat() {
        return readFloat("");
    }

    public float readFloat(String message) {
        String tLinha;

        while (true) {
            try {
                printMessage(message);
                tLinha = readLine().trim();
                if (tLinha.equals(""))
                    return 0.0f;
                return Float.parseFloat(tLinha);
            } catch (NumberFormatException e) {
                System.out.println("Not an float. Please try again!");
            }
        }
    }

    public long readLong() {
        return readLong("");
    }

    public long readLong(String message) {
        String tLinha;

        while (true) {
            try {
                printMessage(message);
                tLinha = readLine().trim();
                if (tLinha.equals(""))
                    return 0L;
                return Long.parseLong(tLinha);
            } catch (NumberFormatException e) {
                System.out.println("Not an long. Please try again!");
            }
        }
    }
}

```

```

public int readInt() {
    return readInt("");
}

public int readInt(String message) {
    String tLinha;

    while (true) {
        try {
            printMessage(message);
            tLinha = readLine().trim();
            if (tLinha.equals(""))
                return 0;
            return Integer.parseInt(tLinha);
        } catch (NumberFormatException e) {
            System.out.println("Not an int. Please try again!");
        }
    }
}

public short readShort() {
    return readShort("");
}

public short readShort(String message) {
    String tLinha;

    while (true) {
        try {
            printMessage(message);
            tLinha = readLine().trim();
            if (tLinha.equals(""))
                return (short) 0;
            return Short.parseShort(tLinha);
        } catch (NumberFormatException e) {
            System.out.println("Not an short. Please try again!");
        }
    }
}

public byte readByte() {
    return readByte("");
}

public byte readByte(String message) {
    String tLinha;

    while (true) {
        try {
            printMessage(message);
            tLinha = readLine().trim();
            if (tLinha.equals(""))
                return (byte) 0;
            return Byte.parseByte(tLinha);
        } catch (NumberFormatException e) {
            System.out.println("Not an byte. Please try again!");
        }
    }
}

```

```

public Date readDate() {
    return this.readDate("", "dd/MM/yyyy");
}

public Date readDate(String message) {
    return this.readDate(message, "dd/MM/yyyy");
}

public Date readDate(String message, String format) {
    dateFormatter.applyPattern(format);
    String tLinha;

    while (true) {
        try {
            printMessage(message);
            tLinha = readLine().trim();
            return dateFormatter.parse(tLinha);
        } catch (ParseException e) {
            System.out.println("Data invalida. Tente novamente!");
        }
    }
}

public boolean readBoolean() {
    return Boolean.parseBoolean(this.readString());
}

public boolean readBoolean(String message) {
    return Boolean.parseBoolean(this.readString(message));
}

/**
 * Lê uma data no formato dd/MM/yyyy e retorna objeto LocalDate
 *
 * @param mensagem
 * @return
 */
public LocalDate readLocalDate(String mensagem) {
    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    String tLinha = "";
    while (true) {
        try {
            printMessage(mensagem);
            tLinha = readLine().trim();
            return LocalDate.parse(tLinha, dtf);
        } catch (DateTimeParseException e) {
            System.out.println("Data invalida. Tente novamente!");
        }
    }
}
}

```